

Windows Essentials

Date: 09/11/2026

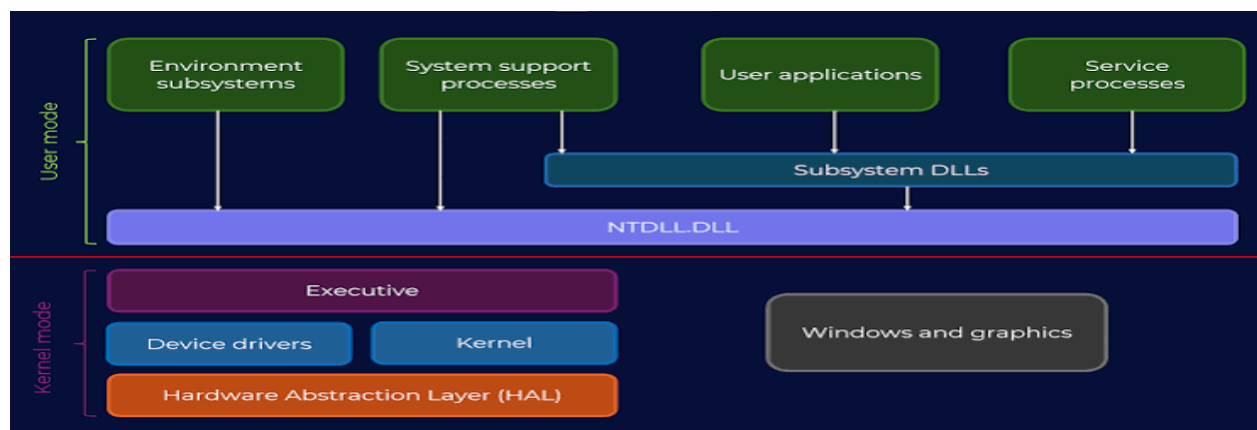
Windows Architecture:

Windows Architecture

- Windows architecture is the internal design that lets applications, services, drivers, and hardware work together safely. The central idea is **separation of privilege** ordinary programs run with limited access, while the operating system's trusted core runs with full access to the machine. Windows mainly uses two processor modes: **user mode** and **kernel mode**
- When you open Chrome, run Python, connect a USB device, or read a file, Windows manages the request in the background. Modern Windows is based on the Windows NT architecture, which separates normal applications from the protected operating-system core

Main Architecture:

- Windows architecture is divided mainly into **User Mode** and **Kernel Mode**.



- This separation is important because user-mode programs have restricted access, while kernel-mode components can access system memory, CPU instructions, and hardware resources

User Mode:

User mode is where normal applications run. Examples include browsers, Microsoft Office, games, PowerShell, VS Code, and Python programs.

A user-mode program cannot directly:

- Access physical memory.
- Execute privileged CPU instructions.
- Control disk or network hardware directly.
- Read or modify another process's memory without permission.
- Load arbitrary kernel code.

Instead, applications use Windows APIs. Common DLL files include:

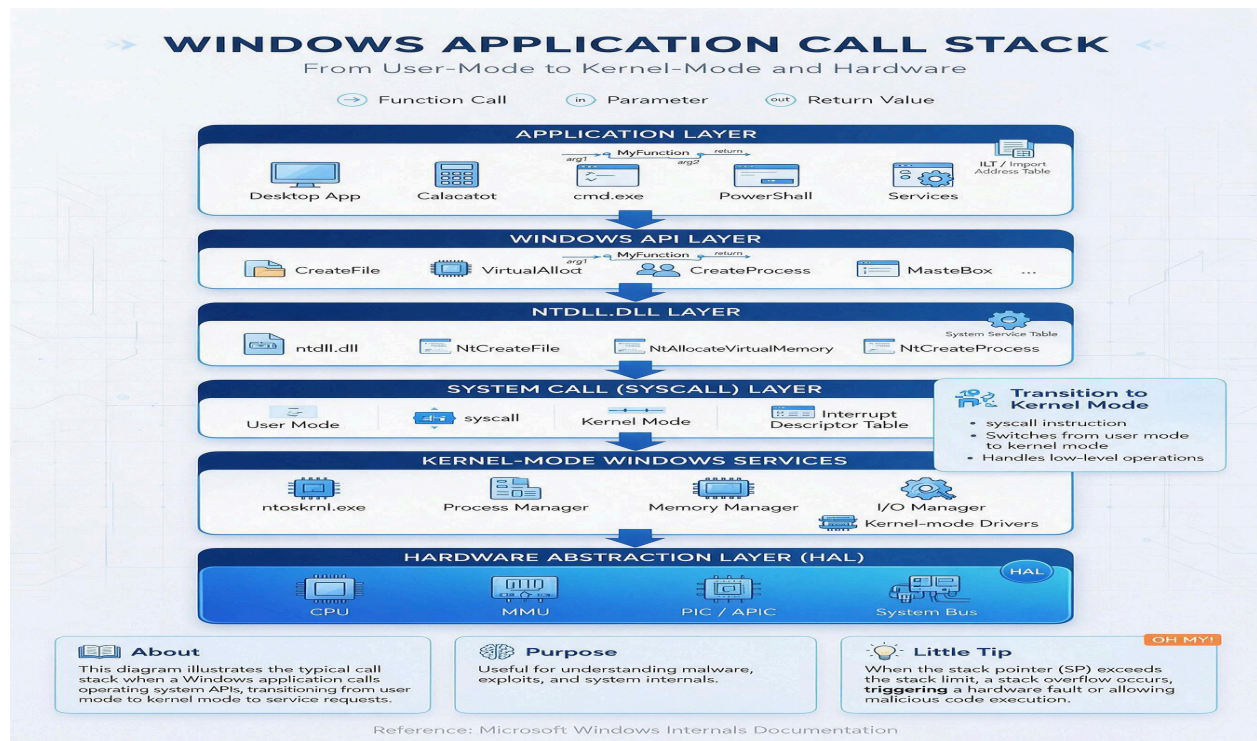
- `Kernel32.dll` for files, processes, threads, and memory-related functions.
- `User32.dll` for windows, buttons, keyboard, mouse, and GUI messages.
- `Advapi32.dll` for security, services, registry, and user accounts.
- `Ws2_32.dll` for network sockets.
- `Ntdll.dll` for lower-level native API functions and system-call transition support.

For example, when a Python program opens a file, it eventually requests Windows to perform the operation. The application does not communicate with the disk controller itself.

System Calls:

- A **system call** is the controlled transition from user mode to kernel mode.
- The CPU changes privilege level during this transition. Windows validates the request, checks access rights where needed, and passes the work to the relevant

internal component or driver. This boundary prevents ordinary programs from having unrestricted control over the system.



Kernel Mode:

- Kernel mode contains Windows components that need full authority over the computer. A faulty application in user mode normally affects only itself. A faulty kernel driver can crash or compromise the full operating system because it has privileged access.

The main kernel-mode components are:

- **Windows Executive**
- **Windows Kernel**
- **Device Drivers**
- **Hardware Abstraction Layer**

Windows Executive:

The **Executive** provides high-level core operating-system services. It is part of the kernel-mode Windows core and is responsible for managing major system resources.

Important Executive managers include:

- **Object Manager:** Represents system resources as objects, such as files, processes, threads, events, mutexes, and registry keys. It also manages handles used by programs to refer to these objects.
- **Process Manager:** Creates and terminates processes and threads.
- **Virtual Memory Manager:** Gives each process its own virtual address space and maps virtual memory to physical RAM.
- **I/O Manager:** Sends input/output requests between applications, file systems, drivers, and devices.
- **Security Reference Monitor:** Enforces access checks using security tokens, permissions, and access-control lists.
- **Configuration Manager:** Manages the Windows Registry.
- **Cache Manager:** Improves performance by keeping frequently used file data in memory.
- **Plug and Play Manager:** Detects and configures connected hardware.
- **Power Manager:** Coordinates sleep, hibernation, battery use, and device power states.

Windows Kernel:

The Windows Kernel handles the lowest-level operating-system work. Its responsibilities include:

- Thread scheduling.
- Interrupt handling.
- Exception handling.
- Synchronization.

- Multiprocessor coordination.

Windows schedules **threads**, not complete processes. A process is a running program with its own resources and address space. A thread is the actual execution path inside that process.

For example, Chrome can run many threads at once: one thread for the user interface, others for tabs, networking, rendering, and background tasks.

Device Drivers

A driver is software that allows Windows to communicate with a particular hardware device or software component. Drivers run mainly in kernel mode because they must interact closely with hardware. Windows uses low-level drivers beneath the operating system kernel to support devices.

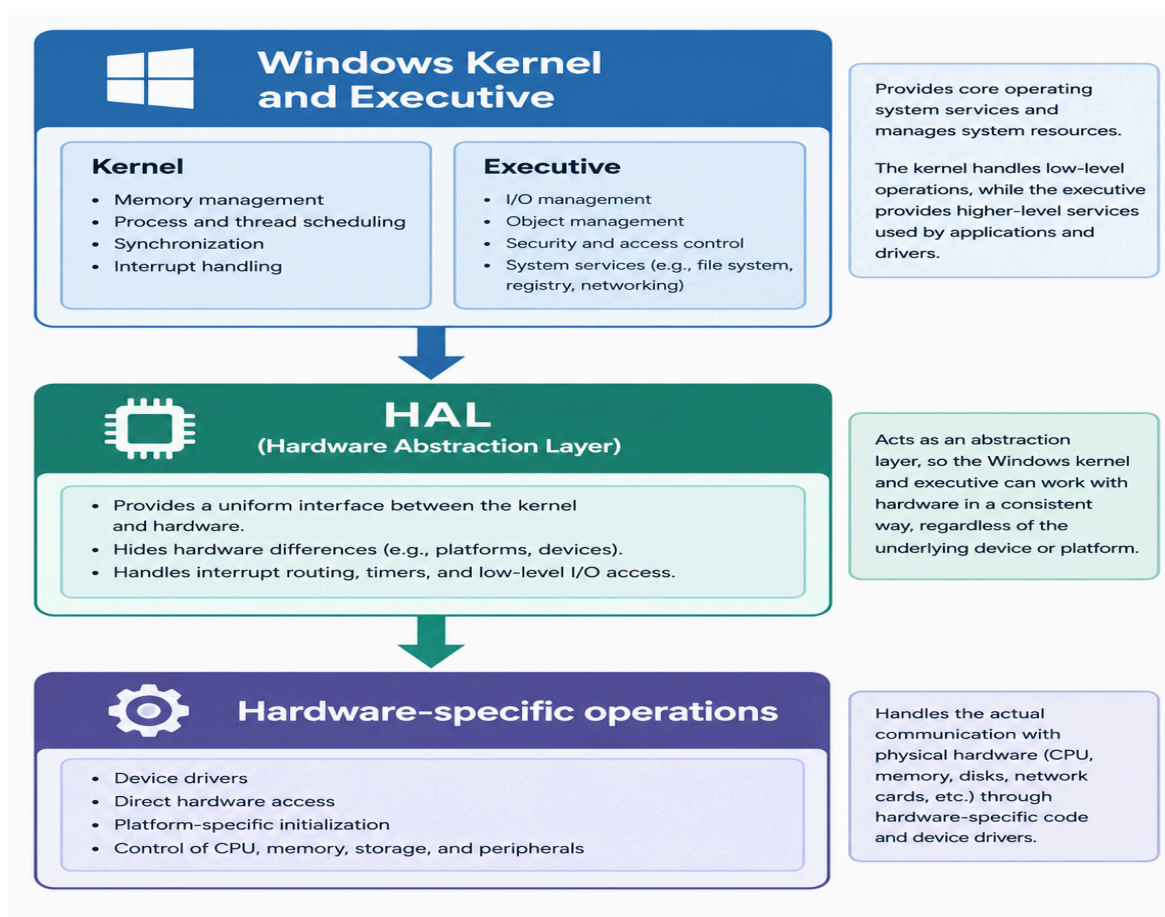
Examples include:

- Storage drivers for SSDs and hard disks.
- Network drivers for Wi-Fi and Ethernet adapters.
- GPU drivers for graphics cards.
- USB drivers for keyboards, mice, and external devices.
- File-system drivers such as NTFS.
- Audio drivers.

A driver commonly receives an I/O request packet from the I/O Manager, processes it, and may pass it down to another driver in a stack until the request reaches hardware.

HAL:

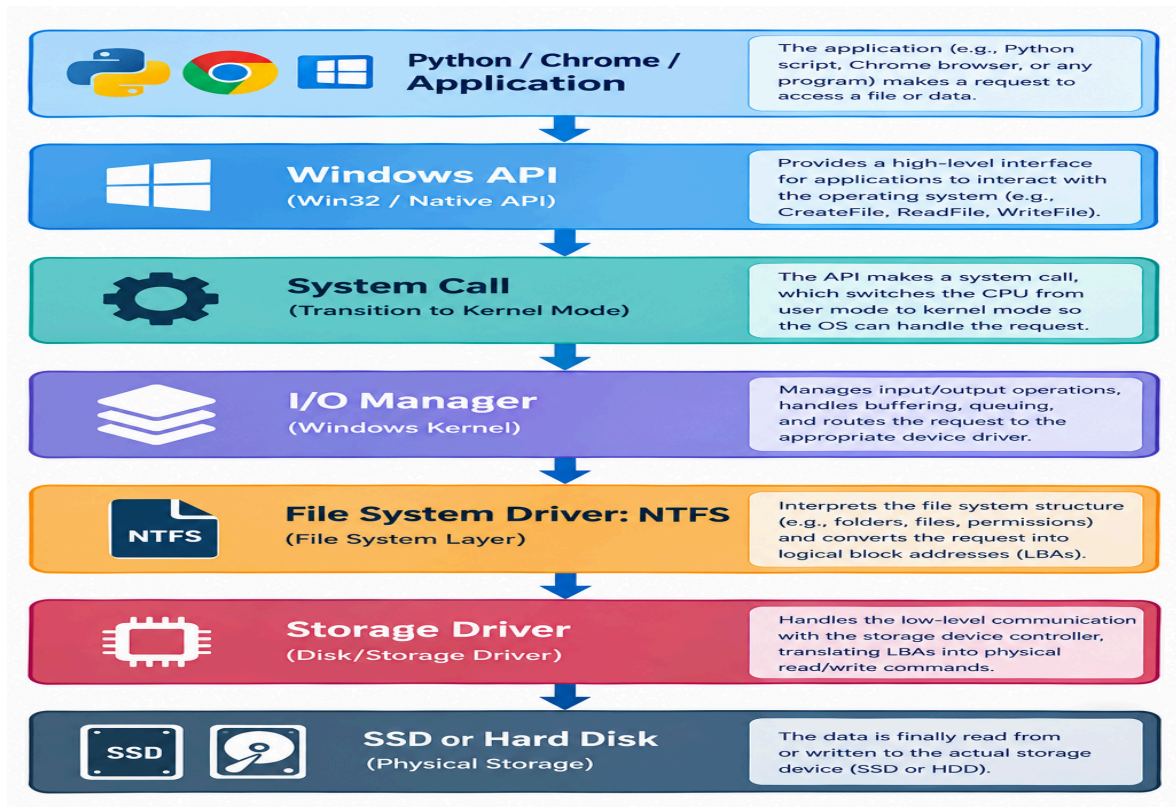
The **Hardware Abstraction Layer**, or HAL, sits between Windows and hardware-specific details. It lets the rest of Windows work across different motherboard, processor, interrupt-controller, and platform designs without needing to know every hardware implementation



- The HAL reduces dependency on one exact hardware design.

File Access Flow:

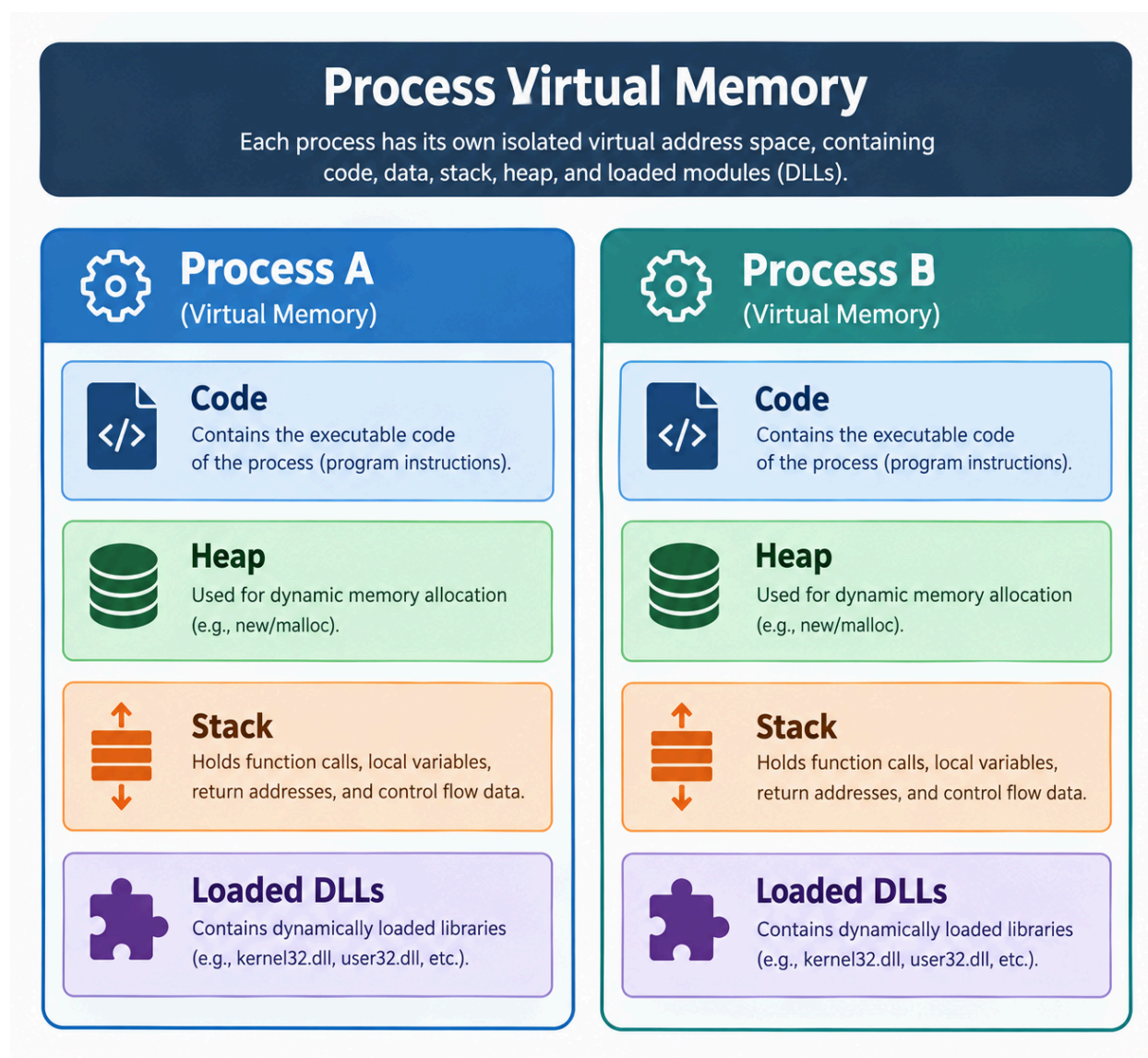
- When an application reads a file, the request usually follows this path:



- At each stage, Windows can verify permissions, manage caching, queue the request, and return errors safely. This is why an application cannot simply read any file from the disk without authorization.

Memory Architecture:

- Windows uses **virtual memory**. Each process receives a private virtual address space. This means one application normally cannot access another application's memory directly.



The Memory Manager maps virtual addresses to physical RAM. If RAM is limited, Windows can move less-active memory pages to disk-based storage. Memory protection is a major reason why a crash in one ordinary application does not automatically crash every other program.

Security Relevance:

For cybersecurity, the boundary between user mode and kernel mode is critical.

- **User-mode malware** runs with the permissions of the compromised user or process. It is restricted by process isolation and Windows security controls.

- **Kernel-mode malware or a malicious driver** has much greater access to processes, memory, security controls, and hardware.
- **Driver vulnerabilities** are high impact because an attacker may use them to gain kernel-level privileges.
- **System calls** are a useful area for malware analysis, EDR research, debugging, and Windows internals study because they connect user-mode activity with kernel-mode services.

Windows architecture is therefore not only an operating-system topic. It is the foundation for Windows malware analysis, reverse engineering, exploit development, incident response, and defensive monitoring

-

Users and Groups in Windows:

In Windows, users and groups are the basic building blocks of identity and access control. A user represents a person, service, or automated account that can log on and run programs. A group is a collection of users, computers, or other groups that can be managed as a single unit for permissions and rights. This design allows administrators to assign access once to a group instead of repeating the same configuration for many individual users.

Security Identifiers and the SAM Database

Every user and group in Windows is identified by a **Security Identifier**, or **SID**. A SID is a unique value that Windows uses internally to represent that security principal. Even if you rename a user account, its SID remains the same, and Windows continues to use that SID for permissions and access checks.

Local users and groups are stored in the **Security Accounts Manager** database, commonly called **SAM**. The SAM is a protected system database located at `%windir%\System32\config\SAM` and is also exposed in the registry under `HKEY_LOCAL_MACHINE\SAM`. It contains local account names, hashed passwords, SIDs, and group memberships for the local machine. On a domain, similar information is

stored in **Active Directory** on domain controllers, but the core concept of SIDs and group-based permissions remains the same

Local Users and Local Groups

On a standalone Windows system, you work mainly with **local users** and **local groups**. A local user account exists only on that specific machine. Examples include the built-in Administrator account, the built-in Guest account, and any additional accounts created through Settings, Computer Management, or PowerShell.

Local groups are used to organize local users and sometimes domain accounts when the machine is joined to a domain. Common built-in local groups include:

- **Administrators:** Members have full control over the system.
- **Users:** Standard accounts with limited privileges.
- **Power Users:** Legacy group with more rights than standard users but fewer than administrators.
- **Guests:** Very limited access, often disabled by default.

You can also create custom local groups and assign them specific permissions on files, folders, registry keys, or local policies. This is the practical way to implement the principle of least privilege: instead of giving many permissions to each user individually, you assign those permissions to a group and then place users into that group

Domain Users and Domain Groups

In an Active Directory environment, Windows uses **domain users** and **domain groups**. These accounts are stored centrally on domain controllers and can be used across all systems in the domain.

Domain groups are divided mainly into:

- **Global groups:** Typically contain user accounts from the same domain. They are used to organize users by role, department, or function.
- **Domain local groups:** Used to assign permissions to resources on a specific domain member, such as file servers or application servers. They can contain global groups, other domain local groups, and users.

- **Universal groups:** Can contain users and groups from any domain in the forest and are used in larger, multi-domain environments.

A common design pattern is “AGDLP”: users go into **A**ccounts (global) groups, those groups go into **D**omain **L**ocal **P**ermissions groups, and permissions are assigned to the domain local groups on resources. This keeps management scalable and clear

Access Tokens and Logon

When a user logs on to Windows, the **Local Security Authority** creates an **access token** for that session. The access token is a kernel object that represents the user’s identity and privileges for that logon. It contains:

- The user’s SID.
- The SIDs of all groups the user belongs to.
- A list of privileges and rights assigned to the user and their groups.
- Additional security information such as integrity level and restricted SIDs in some cases.

Every process started by that user inherits a copy of this access token. When a thread in that process tries to access a securable object, such as a file, registry key, or named pipe, Windows compares the token’s SIDs and privileges against the object’s security descriptor and access control lists. If the token does not contain the required SID or privilege, access is denied. This mechanism is central to Windows security and is why running a program as Administrator can change what that program is allowed to do

Privileges, Rights, and Permissions

Windows distinguishes between **user rights**, **privileges**, and **permissions**:

- **User rights** control system-level actions, such as “Log on locally,” “Shut down the system,” or “Debug programs.”
- **Privileges** are specific capabilities attached to a token, such as `SeDebugPrivilege` or `SeBackupPrivilege`.
- **Permissions** are access rules on objects, such as Read, Write, Modify, or Full Control on a file or folder.

Groups simplify this model. Instead of assigning complex permissions and rights to each user, administrators assign them to groups and then manage group membership. When a user is added to or removed from a group, their effective access changes automatically at the next logon, because their access token is rebuilt with the updated group SIDs

Practical Relevance for Security and Internals

For cybersecurity and Windows internals work, users and groups matter in several ways:

- **Privilege escalation** often involves moving from a standard user token to an administrative or system-level token.
- **Token manipulation** is a common technique in malware and post-exploitation tools, where processes are spawned with different tokens or privileges.
- **Access control analysis** requires understanding how SIDs, groups, and ACLs interact to allow or deny access to files, registry keys, and other objects.
- **Incident response** uses user and group information to determine which accounts were used, what privileges they had, and what resources they could access.

Tools such as `whoami`, `net user`, `net localgroup`, PowerShell cmdlets like `Get-LocalUser`, `Get-LocalGroup`, and advanced utilities like TokenUniverse help inspect and experiment with users, groups, and access tokens in a controlled lab environment

Creating and Managing User Accounts

Creating and managing user accounts can be performed in various ways, including through the Windows GUI, Command Prompt, and PowerShell. Below are examples of each method.

1. Using the Windows GUI

To create a new user account via the GUI:

1. Open the **Control Panel**.

2. Select **User Accounts**.
3. Click on **Manage another account**.
4. Choose **Add a new user in PC settings**, and then follow the prompts.

2. Using Command Prompt

For developers comfortable with command line interfaces, you can create a user account using the `net user` command:

```
net user username password /add
```

For example, to create a user named “devuser” with the password “DevPassword123”, use:

```
net user devuser DevPassword123 /add
```

3. Using PowerShell

PowerShell provides a more versatile option for managing user accounts:

```
New-LocalUser -Name "devuser" -Password (ConvertTo-SecureString  
"DevPassword123" -AsPlainText -Force)
```

This command not only creates the user but also securely converts the password into a format that Windows can utilize.

Creating and Managing Groups

Just as with user accounts, groups can be managed through various methods:

1. Using the Windows GUI

To create a new group via the GUI:

1. Open **Computer Management**.
2. Navigate to **Local Users and Groups > Groups**.
3. Right-click on the **Groups** folder and select **New Group**.
4. Enter a group name and add members to the group.

2. Using Command Prompt

You can also create a new group via the Command Prompt:

```
net localgroup groupname /add
```

For instance, to create a group named “Developers”:

```
net localgroup Developers /add
```

3. Using PowerShell

PowerShell offers a powerful syntax to manage groups:

```
New-LocalGroup -Name "Developers" -Description "Group for  
development team"
```

To add a user to a group, you can use:

```
Add-LocalGroupMember -Group "Developers" -Member "devuser"
```

Permissions and Security

Understanding permissions is crucial for securing applications and data. Permissions in Windows can be assigned at the file system level or as group policies. Here's a closer look:

File System Permissions

Windows uses Access Control Lists (ACLs) to define permissions on files and folders. Each user and group can have specific permissions (like Read, Write, Modify) based on their role in the system.

Example of Setting File Permissions

To modify permissions via PowerShell:

```
icacls "C:pathtofile" /grant "Developers":(R,W)
```

This command grants the “Developers” group Read and Write permissions to the specified file or folder.

Best Practices for User and Group Management

To ensure your Windows environments remain secure and manageable, consider the following best practices:

1. **Limit Admin Accounts:** Use standard user accounts for day-to-day tasks to reduce the risk of malware infections.
2. **Regular Audits:** Regularly review user accounts and group memberships to remove any unnecessary access.

3. **Implement Least Privilege:** Always assign the minimum necessary permissions to accomplish a task.
4. **Use Descriptive Group Names:** This practice simplifies user management and improves clarity in permissions.

Conclusion

Effective user and group management in Windows is essential for developers and system administrators alike. By mastering the tools and techniques discussed in this article, you'll be better equipped to manage user access securely and efficiently. As you implement these practices, your environment will not only be more organized but also significantly more secure.

Whether you're managing user access for a small team or a large enterprise, these insights aim to empower your workflow, enabling you to focus on what you do best—developing innovative software solutions!

File Permissions in Windows:

File permissions in Windows define who can access a file or folder and what actions they are allowed to perform. These permissions are enforced by the file system, primarily NTFS, and are a core part of Windows security. Every file and folder on an NTFS volume has a **security descriptor** attached to it. The security descriptor contains the owner, primary group, a **Discretionary Access Control List (DACL)**, and optionally a **System Access Control List (SACL)**. The DACL is the part that actually controls access

Security Descriptor and ACLs

A **security descriptor** is a data structure that Windows uses to store security information for a securable object, such as a file, folder, registry key, or named pipe. It includes:

- **Owner SID:** Identifies the user or group that owns the object.
- **Primary Group SID:** Used mainly for POSIX compatibility and some legacy scenarios.
- **DACL:** Lists which users and groups are allowed or denied specific access rights.
- **SACL:** Specifies which access attempts should be logged for auditing.

The DACL is made up of **Access Control Entries (ACEs)**. Each ACE names one trustee (a user, group, or computer account) and specifies a set of permissions, along with whether those permissions are allowed or denied. When a process tries to access a file, Windows compares the process's access token against the file's DACL to decide whether to grant or deny the request.

NTFS Permission Types

NTFS defines several standard permission levels that combine lower-level access rights. Common standard permissions include:

- **Read:** View file contents and basic attributes.
- **Write:** Modify file contents and certain attributes.
- **Read & Execute:** Read the file and, if it is an executable, run it.
- **Modify:** Read, write, delete, and execute the file.
- **Full Control:** All possible permissions, including changing permissions and taking ownership.

Each of these standard permissions maps to a set of more granular **special permissions**, such as "Read data," "Write data," "Append data," "Delete subfolders and files," "Change permissions," and "Take ownership." Advanced users and administrators can view and edit these special permissions through the Advanced Security Settings dialog or command-line tools

Allow and Deny Entries

Every permission entry in a DACL can be either **Allow** or **Deny**. The general rules are:

- **Deny entries take precedence over Allow entries at the same level.** If a user or one of their groups has a Deny ACE for a specific right on the same object where another ACE allows that right, the Deny wins
- **Explicit permissions take precedence over inherited permissions.** An ACE set directly on the object (explicit) overrides an inherited ACE from a parent folder, even if the inherited ACE is a Deny and the explicit ACE is an Allow.

This means that a single Deny ACE can block access even when multiple Allow ACEs would otherwise grant it, but an explicit Allow on a child object can override an inherited Deny from a parent.

Inheritance

Permissions can be assigned directly on an object (**explicit**) or inherited from a parent folder (**inherited**). When you set permissions on a folder, you can configure them to propagate to:

- Files within the folder.
- Subfolders within the folder.
- Both files and subfolders.

Inheritance flags such as **Object Inherit (OI)** and **Container Inherit (CI)** control this behavior. Inherited permissions simplify management because you can set a baseline on a parent folder and let child objects automatically receive those permissions. However, inheritance can be blocked on specific files or subfolders if needed, which converts inherited entries into explicit entries on that object or removes them entirely.

Effective Permissions

A user's **effective permissions** on a file or folder are the result of combining all applicable Allow and Deny entries from:

- The user's own ACEs.
- ACEs for all groups the user belongs to.
- Explicit and inherited entries along the folder path.

Windows calculates effective access by:

1. Collecting all relevant ACEs for the user and their groups.
2. Ordering them by type: explicit deny, explicit allow, inherited deny (nearest parent first), inherited allow (nearest parent first).
3. Applying Deny entries before Allow entries at each level.
4. Producing the final set of allowed rights.

The **Effective Access** tab in Advanced Security Settings performs this calculation automatically. You select a user or group, and Windows shows the resulting permissions after considering inheritance, group membership, and precedence rules.

Share Permissions vs NTFS Permissions

When accessing files over a network share, two layers of permissions apply:

- **Share permissions:** Control access at the share level (Read, Change, Full Control).
- **NTFS permissions:** Control access at the file and folder level on the disk.

Windows applies both sets of permissions and uses the **most restrictive** result. For example, if NTFS permissions grant Full Control but share permissions only allow Read, the user effectively has Read-only access over the network. Best practice is to set share permissions broadly (often “Everyone – Full Control”) and enforce detailed access control through NTFS permissions.

Practical Tools

Common tools for working with file permissions include:

- **File Explorer:** Right-click a file or folder → Properties → Security tab → Advanced.
- **icacls:** Command-line tool to view and modify ACLs.
- **Get-Acl / Set-Acl:** PowerShell cmdlets for scripting permission changes.

- **Effective Access** UI: Built into Advanced Security Settings to test what a specific user can do.

These tools are useful for learning, auditing, and designing permission structures in a lab or production environment.

How to check file permissions

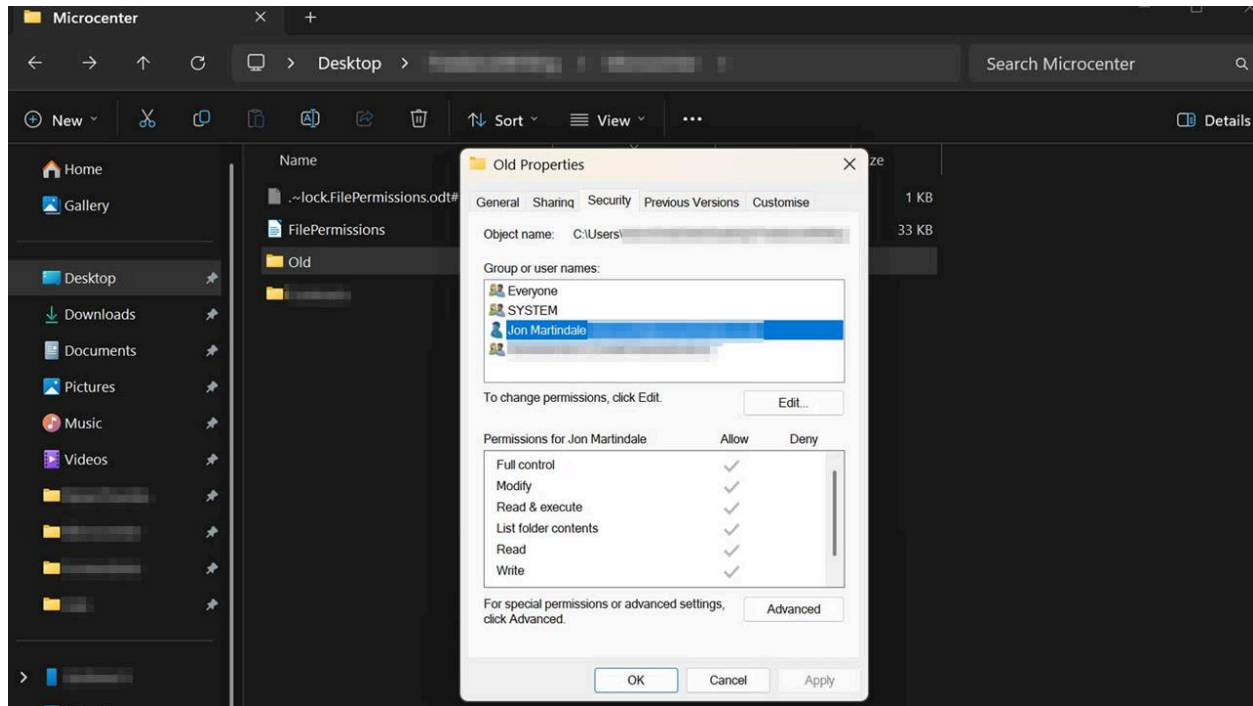


Photo: Microsoft/Jon Martindale

Before you can edit the permissions for a file or folder, you need to see what the current permissions are first. Find the file that you want to check, then follow these steps to see its standing permissions.

- Right-click or tap and hold on the file or folder, and select Properties.
- Select the Security tab in the top row.
- In the top section called “Group or user names,” select individual accounts to view their particular permissions in the bottom section called, “Permissions for [Account name]”.

- Each account will have a check in either Allow or Deny, depending on the permissions for that particular account for each of the respective permissions.

How to give another user access to a file or folder

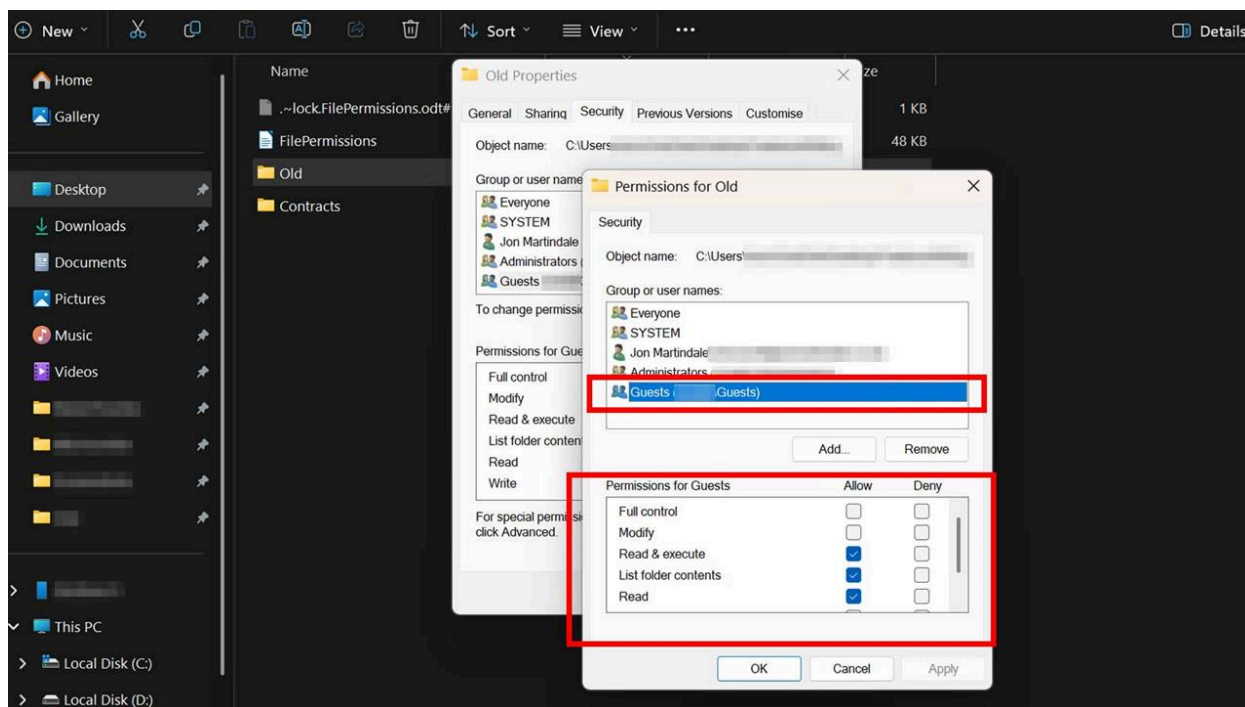


Photo: Microsoft/Jon Martindale

To give another user access to a file or folder, you need to adjust the permissions on that particular file or folder, or its parent folder. Here's how to do it.

- Right-click or tap and hold on the file or folder, and select Properties.
- Select the Security tab in the top row.
- Select Edit.
- Select the group or user to change permissions for. In the above example, we're modifying "Guests."

- Under the heading, “Permissions for [AccountName],” use the check boxes within the “Allow” column to decide what level of access and permissions to give a user. Remember that some types of permission grant specific access – full control doesn’t need to be given and access can be restricted.
- Once permissions have been adjusted, select Apply, then OK to confirm the changes, then OK again on the main “Properties,” window.

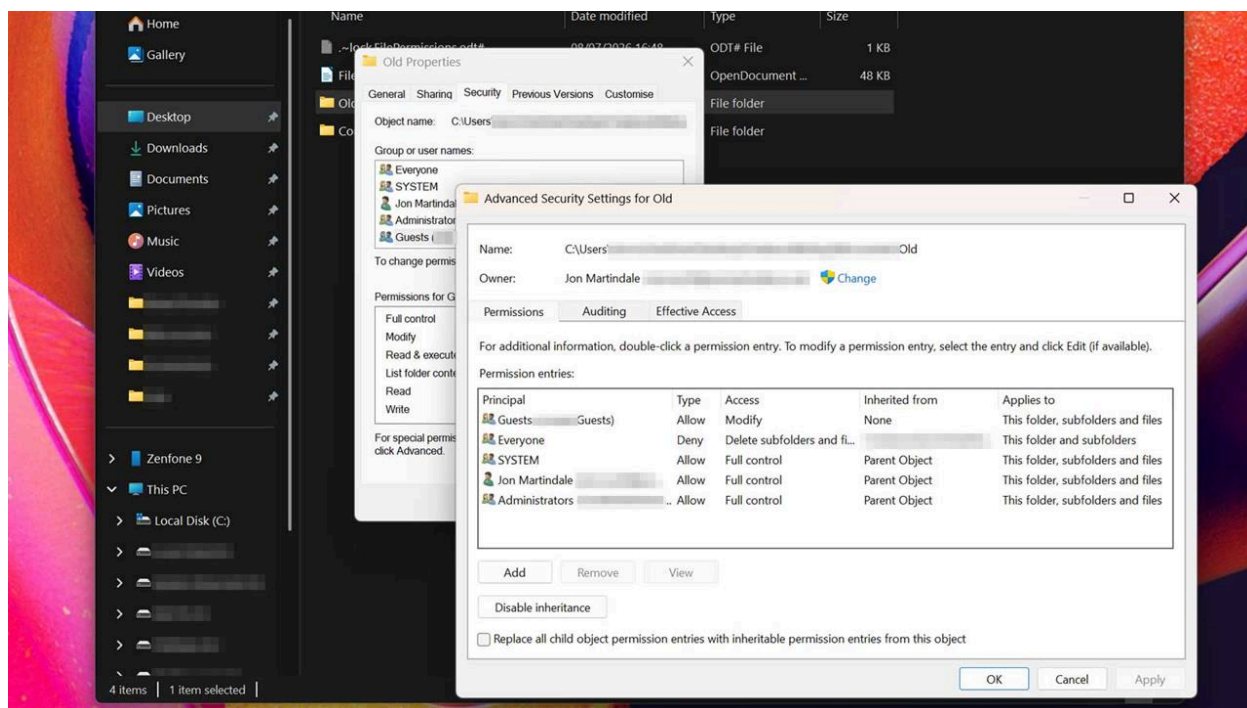
How to remove a user’s access to a file or folder

Removing a group or user’s access to a file or folder is very much the same as adding a new user, only unchecking Allow boxes, and specifically checking Deny boxes.

- Right-click or tap and hold on the file or folder, and select Properties.
- Select the Security tab in the top row.
- Select Edit.
- Uncheck any relevant Allow boxes, such as Full Control, and Modify. If the user or group needs to still be able to see the folder, or read its contents without being able to edit it, consider leaving Read or List folder contents checked.
- Since it’s possible for a user to inherit access or edit permissions from another group, it’s important to explicitly Deny access if this user should not be able to access this file or folder. To remove access entirely, check every Deny box for that user. Alternatively, for more nuance, only select the most relevant boxes to leave, for example, read access intact.

- Once permissions have been adjusted for this specific file or folder from the relevant user accounts or groups, select Apply, followed by OK, then select OK again.

How to turn off permission inheritance for a file or folder



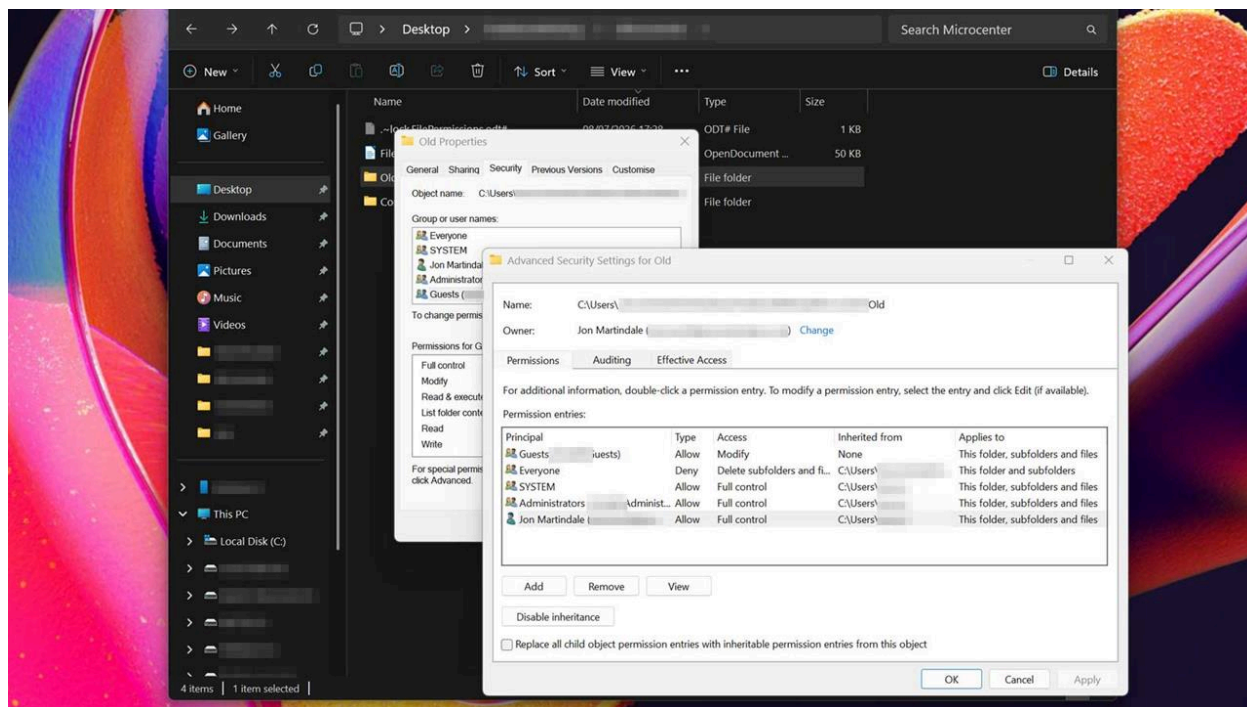
Source: Microsoft/Jon Martindale

Turning off permission inheritance is a straightforward way to restrict access to a file or sub-folder within a project or nest of files or folders without breaking off access to everything above it in the chain. This can be particularly useful when bringing new users on to a collaborative project, but you aren't keen on them having access to everything and don't want to control access individually for every user.

- Right-click or tap and hold on the file or folder, and select Properties.
- Select the Security tab in the top row.

- Select Advanced.
- Select Disable inheritance.
- Decide whether to convert all existing inherited permissions for this file or folder into explicit permissions, or to remove those inherited permissions entirely. Select the corresponding option.
- To re-enable inheritance for that file or folder, repeat the steps above, and select Enable inheritance. Follow any onscreen instructions to restore the ability to inherit permissions.

How to change the owner of a file or folder



Source: Microsoft/Jon Martindale

If you want to hand a file or folder off to another user and grant them the ultimate control over it and the ability to control who else can access it (including you), then you need to change the owner.

Note: The new owner may still not have much in the way of permissions to read or edit the file, but with ownership, they will be able to change that themselves.

- Make sure you are signed in to an administrator account.
- Right-click or tap and hold on the file or folder, and select Properties.
- Select the Security tab in the top row.
- Select Advanced.
- Next to where it says "Owner:" followed by your username and email, select Change. If it asks for admin approval, give it. If you need to save existing

- Type the name of the user or group you want to transfer ownership of the file or folder to. Select Check Names to confirm Windows recognizes it. If it doesn't, make sure you have the name right. If in doubt, check back on the Advanced Security window for the list of users.
- When you're happy with your changes, select OK. Then select Apply, followed by OK, then OK again on the Properties window.

[illegible]

If you make a mistake, lose track of your folder permissions, or find that you've locked yourself out of making further changes, then resetting the permissions to their Windows default is a good way to get back to a baseline so you can start again.

- Use Windows search and type in CMD.
- Right-click or tap-and hold on the Command Prompt result, and select Run as administrator.
- Access the folder you want to change permissions for. Do so by typing in “cd” followed by the folder location, for example: “cd C:\Users\[username]\Desktop\FolderName\Micro Center.” If you need to select a different drive letter, be sure to navigate to that drive first, or include the “/d” command following the cd. For example: “cd /d D:\Users\username\...”
- To reset folder permissions to their Windows defaults, type the following, and press Enter: `icacls "file name" /reset /t /c /l`.

The `icacls` command will open up file and folder permissions, while `/reset` will reset permissions for the folder to their inherited defaults. The `/t` will then apply that same reset to all files and subfolders, while the `/c` command will continue even if there are some files within that produce errors when changing permissions. The `/l` command ensures the changes are applied to the links between files if Windows encounters a shortcut, rather than the file it points to.

You can perform these same Command Prompt permission resets on individual files, too, just supply the full location path of that file, including the file name.

How to make a folder read-only

To make a folder read only, you'll want to return to the Security tab and be extra restrictive with your choices, making sure no users can do anything but read it.

- Right-click or tap and hold on the folder, and select Properties.
- Select the Security tab in the top row.
- Select Edit.

-
- To ensure everyone can read the file, select the Everyone group, and make sure that Read is allowed on the bottom list.
 - Then for each individual account, select them and remove any Allow checks for anything beyond Read.
 - It's probably a good idea that any admin account (or at least one) retains Full control over the file, so that it can be modified, moved, or deleted as needed. Alternatively, to lock down the folder as much as possible, use the "Deny" column to put more explicit restrictions on its use. Be aware though, this can override other permissions, which can create problems with access if niche cases need greater control over the folder and its contents.

How to fix "You don't currently have permission to access this folder."

If you run into this problem, it's possible your network or system administrator doesn't want you to have access to a certain file or folder. If that may be the case, check with them before making any manual changes.

However, if this message has appeared in error and you need access to that folder, here's how to fix it.

- Sign in to an administrator account.
- Right-click or tap and hold on the file or folder, and select Properties.
- Select the Security tab in the top row.
- Select Advanced.

- The easiest way to get access is to change the folder's owner. If that's not possible, skip down to the Permissions section. Next to where it says "Owner:" select Change. If it asks for admin approval, give it.
- Type the username of whoever is getting permission into the box and select Check Names to confirm it's accurate, then select OK. To take ownership of subfolders, check the box that says Replace owner on subcontainers and objects.
- Select Apply, then OK.
- To change Permissions instead, return to the Security tab and select Edit.
- Select the username from the top list.
- Under the heading, "Permissions for [YourAccountName]," use the check boxes within the "Allow" column to decide what level of access and permissions to give the account.
- When finished, select Apply, then OK to confirm the changes, then OK again on the main "Properties," window.

Services in Windows Explanation

Windows services are background programs that provide system functions or application features without needing a logged-in user. Examples include Windows Update, printing, DNS, antivirus, database servers, and web servers. All services are managed by one central component: the **Service Control Manager (SCM)**, which runs in

How the SCM Starts and Controls Services

The SCM starts very early during boot, launched by `wininit.exe` in Session 0. It reads all service definitions from the registry under:

HKLM\SYSTEM\CurrentControlSet\Services

For each service marked as automatic or system-start, the SCM:

- Checks dependencies (other services that must start first).
- Verifies the account the service should run under.
- Launches the service process or loads the driver.
- Waits for the service to connect back and report its status.

When the SCM wants to start a user-mode service, it creates the process with the configured account (LocalSystem, NetworkService, LocalService, or a custom user). The service process then calls `StartServiceCtrlDispatcher()`, which connects it to the SCM over a protected named pipe. After this connection is established, the SCM sends a “start” command, and the service begins its main work

Service Control Handler and Status Reporting

Every user-mode service must tell the SCM what state it is in. To do this, the service:

1. Registers a **control handler function** using `RegisterServiceCtrlHandler()` or `RegisterServiceCtrlHandlerEx()`. This function will receive control codes from the SCM, such as Stop, Pause, Continue, or custom commands.
2. Calls `SetServiceStatus()` to report its current state to the SCM. The status includes fields like current state, accepted control codes, exit code, and wait hints.

Common service states include:

- `SERVICE_STOPPED`: Service is not running.
- `SERVICE_START_PENDING`: Service is starting.
- `SERVICE_RUNNING`: Service is fully operational.
- `SERVICE_STOP_PENDING`: Service is stopping.
- `SERVICE_PAUSED` / `SERVICE_PAUSE_PENDING`: Service is paused or pausing.

When the SCM sends a control code (for example, Stop), it calls the service’s handler function with that code. The handler must:

- Perform the requested action (stop, pause, continue, etc.).

- Call `SetServiceStatus()` if the state changes (for example, from `RUNNING` to `STOP_PENDING` to `STOPPED`).
- Return quickly so the SCM does not think the service is hung.

Every service must at least accept the `SERVICE_CONTROL_INTERROGATE` code, which asks the service to report its current status. Services can also accept codes like `SERVICE_CONTROL_STOP`, `SERVICE_CONTROL_PAUSE`, `SERVICE_CONTROL_CONTINUE`, and others, depending on their design.

Control Codes and How They Are Used

Control codes are numeric commands that the SCM or a management tool sends to a service. Common codes include:

- `SERVICE_CONTROL_STOP` (1): Tell the service to stop.
- `SERVICE_CONTROL_PAUSE` (2): Tell the service to pause.
- `SERVICE_CONTROL_CONTINUE` (3): Tell a paused service to resume.
- `SERVICE_CONTROL_INTERROGATE` (4): Ask the service to report its status.
- `SERVICE_CONTROL_SHUTDOWN`: Notify the service that the system is shutting down.
- Additional codes for network binding changes, hardware profile changes, power events, and custom vendor-defined commands

When you use `sc stop`, `Stop-Service`, or the Services MMC to stop a service, the tool calls `ControlService()` with `SERVICE_CONTROL_STOP`. The SCM forwards this to the service's handler. The service then moves to `STOP_PENDING`, cleans up, and finally reports `STOPPED`.

Service Types and Execution Models

Windows supports different service types:

Kernel-mode services (drivers):

- `SERVICE_KERNEL_DRIVER`: Low-level device drivers.
- `SERVICE_FILE_SYSTEM_DRIVER`: File system drivers like NTFS.

- These are loaded by the kernel during boot based on their “Start” type (boot, system). They do not use the user-mode service APIs and do not have a service account in the same way.

User-mode services:

- `SERVICE_WIN32_OWN_PROCESS`: Runs in its own process.
- `SERVICE_WIN32_SHARE_PROCESS`: Shares a process (like `svchost.exe`) with other services.
- `SERVICE_INTERACTIVE_PROCESS` (legacy): Could interact with the desktop; now discouraged for security.

User-mode services run in Session 0, isolated from user applications. This design, called **Session 0 Isolation**, prevents services from directly interacting with user desktops and reduces certain attack vectors.

Service Accounts and Security Impact

Each user-mode service runs under a specific account defined by the `ObjectName` registry value. Common accounts:

- **LocalSystem**: Extremely powerful, broad local privileges, can access most resources.
- **NetworkService**: Limited local rights, accesses network as the computer account.
- **LocalService**: Limited local and network rights.
- **Custom user accounts**: Used to restrict the service to only what it needs.

The account determines what the service can do. A service running as `LocalSystem` can read most files, modify many registry keys, and interact with many system components. If that service is compromised, the attacker effectively gains those same privileges. Running services under least-privilege accounts is a key hardening practice.

Service Lifecycle in Detail

A typical user-mode service follows this sequence:

1. **Registration:** The service binary is installed and its registry key is created under `HKLM\SYSTEM\CurrentControlSet\Services`. Tools like `sc create`, `New-Service`, or installers handle this.
2. **Startup decision:** At boot, the SCM reads all service keys, sorts them by dependencies, and decides which to start automatically.
3. **Process creation:** For each service to start, the SCM creates the process with the configured account and environment.
4. **Dispatcher connection:** The service calls `StartServiceCtrlDispatcher()`, connecting to the SCM.
5. **Handler registration:** The service calls `RegisterServiceCtrlHandler()` to set up its control handler.
6. **Status reporting:** The service calls `SetServiceStatus()` to move from `START_PENDING` to `RUNNING`.
7. **Main work:** The service performs its function (listening on ports, processing jobs, monitoring, etc.).
8. **Control handling:** The SCM or a tool sends control codes; the handler processes them and updates status.
9. **Shutdown:** On stop or system shutdown, the service cleans up and reports `STOPPED`.

Why This Matters for Security and Internals

Services are central to both system operation and security:

- They are a common **persistence mechanism** for malware, because a malicious service can start automatically with high privileges.
- They often run as **LocalSystem** or other powerful accounts, making them high-value targets.
- Their registry keys, binaries, and control flow are important artifacts in **incident response** and forensics.
- Understanding service startup, control handlers, and status reporting is essential for **malware analysis**, **privilege escalation research**, and **defensive monitoring**.

Command-Line (CMD) commands

How to list all Windows services:

```
sc queryex type=service state=all
```

How to list all Windows services (names only):

```
sc queryex type=service state=all | find /i "SERVICE_NAME:"
```

How to list all the *running* Windows services, excluding the *stopped* / *inactive* ones:

```
sc queryex type=service state=active
```

How to list all the *stopped* / *inactive* Windows services, excluding the *running* ones:

```
sc queryex type=service state=inactive
```

How to search for a given Windows service (by name):

```
sc queryex type=service state=all | find /i "SERVICE_NAME: MyServiceName"
```

How to retrieve the status of a given service (by name):

```
sc query MyServiceName
```

PowerShell commands

How to list all Windows services:

```
Get-Service
```

How to list all Windows services (names only):

```
sc queryex type=service state=all | find /i "SERVICE_NAME:"
```

How to list all the *running* Windows services, excluding the *stopped* / *inactive* ones:

```
Get-Service | Where-Object {$_.Status -eq "Running"}
```

How to list all the *stopped* / *inactive* Windows services, excluding the *running* ones:

```
Get-Service | Where-Object {$_.Status -eq "Stopped"}
```

How to search for a specific Windows service:

```
Get-Service | Where-Object {$_.Name -like "**MyServiceName*"}
```

How to retrieve the status of a given service (by name):

```
Get-Service MyServiceName*
```

Windows Registry

- The Windows Registry is a hierarchical database that stores configuration for the operating system, drivers, services, applications, and user settings. Almost every part of Windows reads from or writes to the registry: boot configuration, device drivers, installed software, user preferences, security policies, and many persistence mechanisms used by both legitimate software and malware. In Regedit, the registry appears as a tree of keys and values under a small set of root keys, but internally it is backed by physical files on disk called hives.
- In the logical view, you see five main root keys. HKEY_CLASSES_ROOT (HKCR) holds file associations and COM class registrations and is actually a merged view of HKLM\Software\Classes and HKCU\Software\Classes. HKEY_CURRENT_USER (HKCU) stores settings for the currently logged-in user, such as desktop appearance, application preferences, and per-user startup entries. HKEY_LOCAL_MACHINE (HKLM) contains system-wide configuration for hardware, drivers, services, installed software, and security settings that apply to all users. HKEY_USERS (HKU) contains loaded user profiles, with each user's SID having its own subkey; HKCU is really just a link to the current user's subkey under HKU. HKEY_CURRENT_CONFIG (HKCC) represents the current hardware profile and is mostly used for display and hardware configuration at boot. These root keys look like simple folders in Regedit, but each one maps to physical hive files on disk
- A hive is a registry database file stored on disk. At boot, the Configuration Manager, which is an executive component in the kernel, loads these hive files and maps them into the registry tree. Major HKLM hives live in %SystemRoot%\System32\config. The SAM hive (HKLM\SAM) stores local user accounts and password hashes. The SECURITY hive (HKLM\SECURITY) stores

local security policies and security descriptors. The SOFTWARE hive (HKLM\SOFTWARE) holds installed software information and system-wide application settings, including many persistence locations. The SYSTEM hive (HKLM\SYSTEM) contains driver and service configuration, boot parameters, and control sets. The HARDWARE hive (HKLM\HARDWARE) is volatile and reflects detected hardware at boot; it is not saved to disk. User profile hives include HKU.DEFAULT, which maps to the DEFAULT file and represents the default user profile, and HKU<SID>, which maps to NTUSER.DAT in each user's profile folder and represents that user's HKCU. There is also UsrClass.dat for per-user file associations and COM registrations under HKCU\Software\Classes. This separation between logical keys in Regedit and physical hive files on disk is important for forensics, malware analysis, and recovery

- Inside the registry, keys act like folders and can contain subkeys and values. Values are named as pairs stored under a key. Each value has a name, a type, and data. Common types include REG_SZ for strings, REG_EXPAND_SZ for strings with environment variables, REG_DWORD for 32-bit numbers, REG_QWORD for 64-bit numbers, REG_BINARY for raw binary data, and REG_MULTI_SZ for multiple strings. An example path is HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run. Under this key, each value represents a program that runs at user logon, and malware often adds entries here to persist after reboot.
- At boot, the kernel and Configuration Manager load critical hives such as SYSTEM, SOFTWARE, SAM, SECURITY, and DEFAULT. The SYSTEM hive tells Windows which drivers and services to load, in what order, and with what start type. The SOFTWARE hive provides system-wide application and component configuration. The SAM and SECURITY hives provide local accounts and security policies. At runtime, applications read and write their settings under HKLM for machine-wide configuration or HKCU for per-user configuration. Drivers and services read configuration from HKLM\SYSTEM\CurrentControlSet\Services. Group Policy settings are applied via registry keys under HKLM\Software\Policies and HKCU\Software\Policies. Many system features, including networking, printing, updates, and security tools, rely heavily on registry configuration
- The services database lives under HKLM\SYSTEM\CurrentControlSet\Services. Each service has a subkey with values such as ImagePath (path to the service

binary or driver), Type (kernel driver, file system driver, or user-mode service), Start (when to start: boot, system, automatic, manual, disabled), DependOnService (dependencies), ObjectName (account the service runs under), and FailureActions (what to do on failure). This is the same location described in the Services section. Changing these keys directly affects how services and drivers load, which is why both attackers and defenders focus heavily on this area.

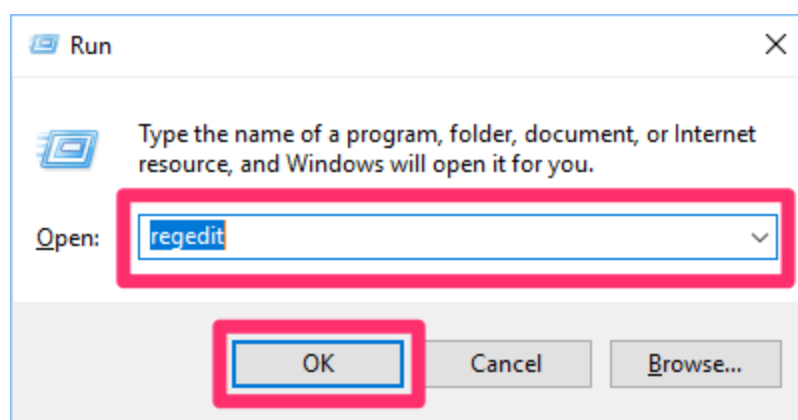
- Registry keys are securable objects. Each key can have a security descriptor with a DACL that controls which users and groups can read, write, delete, or change permissions on that key. Common access rights include KEY_QUERY_VALUE to read values, KEY_SET_VALUE to modify values, KEY_CREATE_SUB_KEY to create subkeys, KEY_DELETE to delete the key, READ_CONTROL and WRITE_DAC to read or modify the key's ACL, and KEY_ALL_ACCESS for full control. Critical hives like SAM, SECURITY, and SYSTEM are protected so that only highly privileged accounts, such as LocalSystem or Administrators with special access, can read or modify them. Some keys are further protected with additional kernel-level restrictions
- From a security perspective, certain registry areas are high-value targets. Run and RunOnce keys under HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run and HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run execute programs at user logon and are commonly abused by malware. Services under HKLM\SYSTEM\CurrentControlSet\Services can be used to add malicious services that start automatically, often with high privileges. Winlogon keys under HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon control logon behavior, shell, and userinit and are critical for system startup. COM and file association keys under HKCR, HKLM\SOFTWARE\Classes, and HKCU\Software\Classes can be abused to execute code when certain files or protocols are used. Group Policy-related keys under HKLM\Software\Policies and HKCU\Software\Policies influence system and user behavior in managed environments. Malware often uses these locations to survive reboots, and defenders monitor them for suspicious changes
- Common tools for working with the registry include regedit.exe as a graphical editor, reg.exe as a command-line tool, and PowerShell with its registry provider (HKLM: and HKCU: drives) and cmdlets like Get-Item, Get-ItemProperty,

Set-ItemProperty, New-Item, and Remove-Item. Sysinternals Autoruns shows many auto-start locations, including registry-based ones. Offline registry editors allow analysis of hives from a dead system or disk image. If you want, the next section can focus on a practical lab: reading and modifying service and Run key entries, observing the effect on startup, and practicing safe registry editing in a VM.

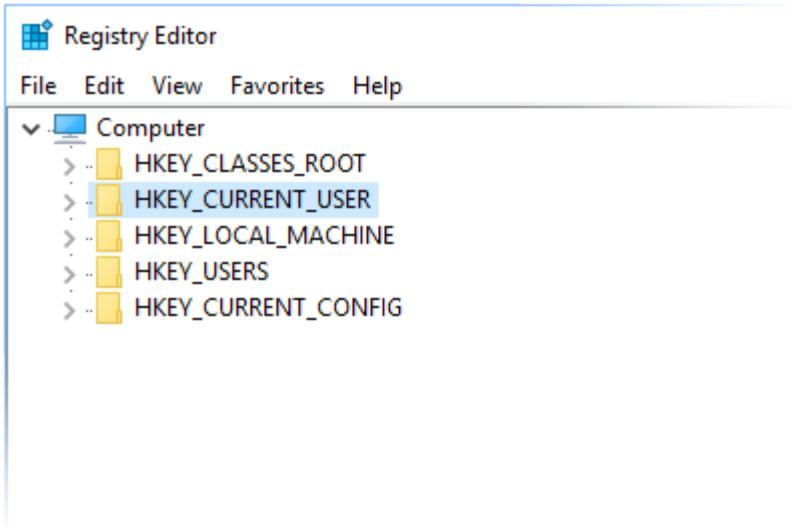
Creating, Editing, or Deleting Registry Keys (Reg Keys) and/or String Values

Open the Registry Editor

1. Open the Windows **Run** dialog box by pressing the **Windows + R** keys (Windows keyboard) or **Command + R** keys (Mac keyboard).



2. In the **Run** dialog box, type **Regedit** and click **OK**.
3. The **Registry Editor** dialog box will open.



The Reg Keys are located within the folders with names that begin with **HKEY_**

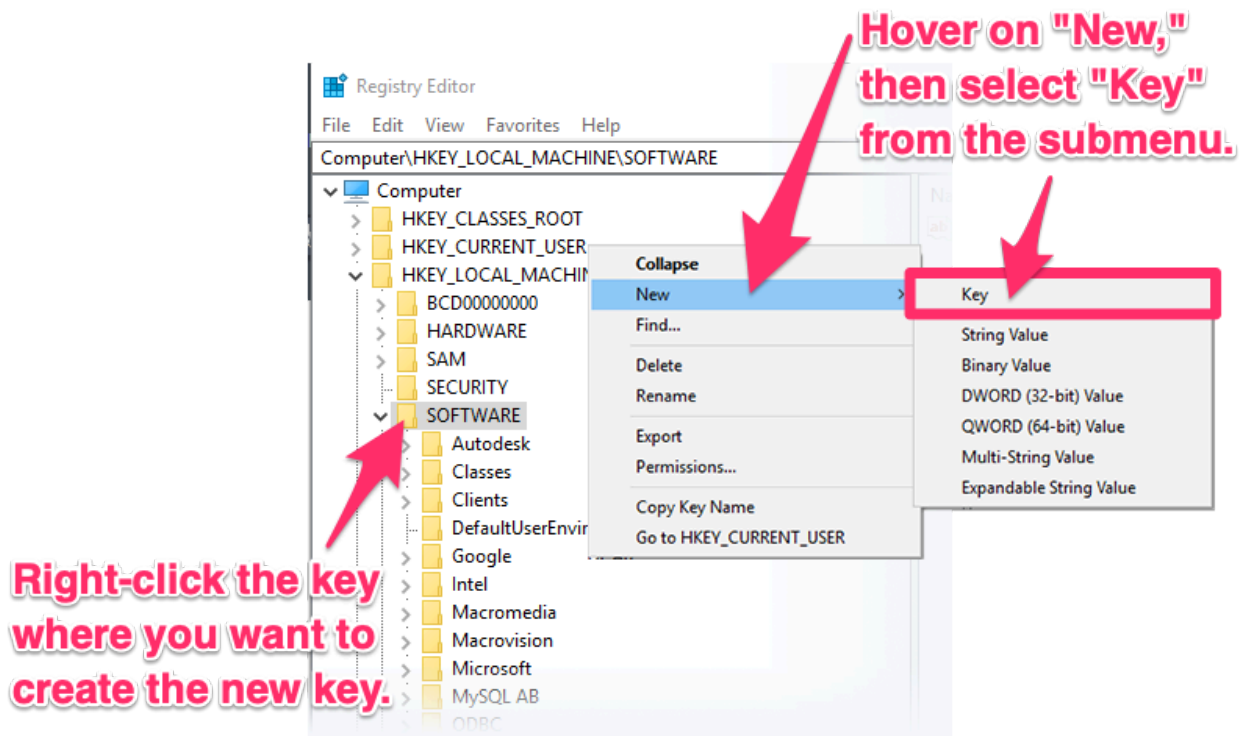
A Reg Key's name consists of its folder path within this folder structure.

Some of our troubleshooting articles will instruct you to [edit](#) or [delete](#) one or more Reg Keys. To locate a Reg Key, expand the first folder in that Reg Key's name (example: **HKEY_LOCAL_MACHINE**) by clicking the right-facing arrow to the left of that folder.

4. Continue expanding folders to locate the Reg Key you need to create, edit, or delete.

Create a Reg Key

To create a new Reg Key, *right-click* the key location where you want to create the new key. Hover the mouse on the **New** option in the menu that opens, then select **Key** from the submenu that opens after that.



In this example, we'll create a key named **Ecograx** in the key location **HKEY_LOCAL_MACHINE\SOFTWARE**

You'll see a new key within the key path you right-clicked

Type a name for the key (example: **Ecograx**).

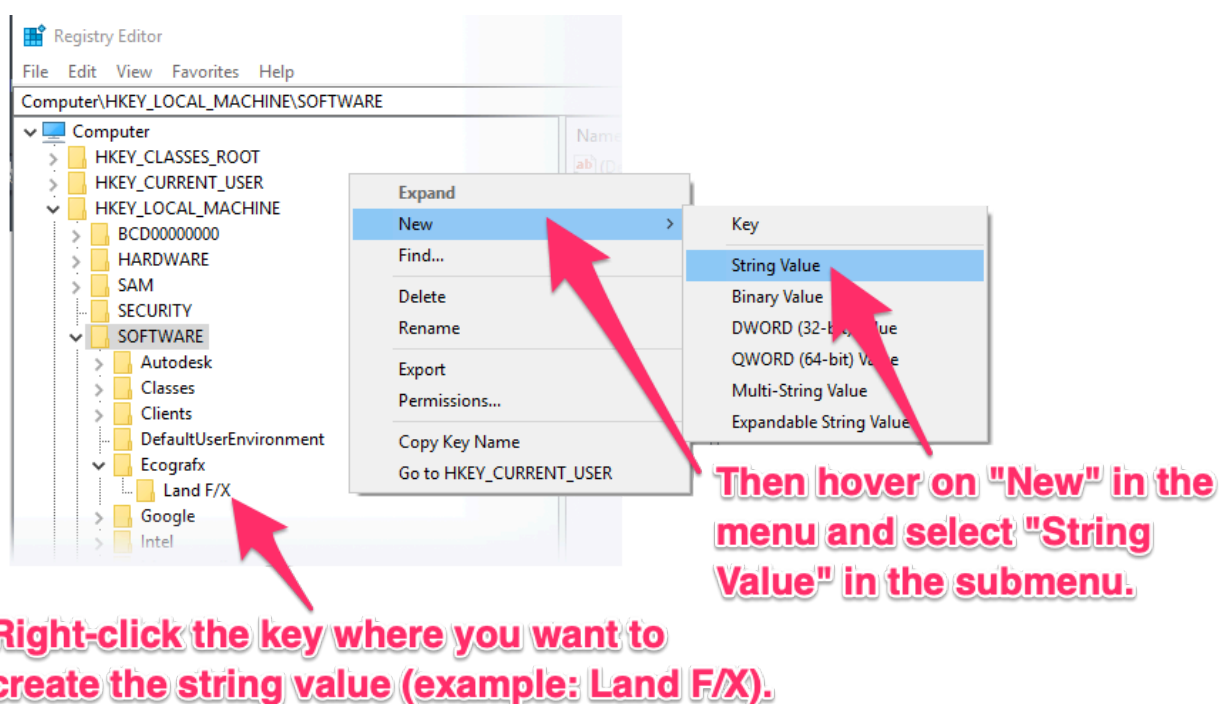
To continue adding keys to the path you've created – for example, if you need to create the key path **HKEY_LOCAL_MACHINE\SOFTWARE\Ecograx\Land F/X** in following our Manual Installation of the Land F/X Workstation steps, repeat these steps, *right-clicking* the new key you just created and selecting the option for the new key in the menu.

If our Knowledge Base or one of our support technicians has instructed you to create a key value, move on to the Create a String Value steps below once you've created the necessary

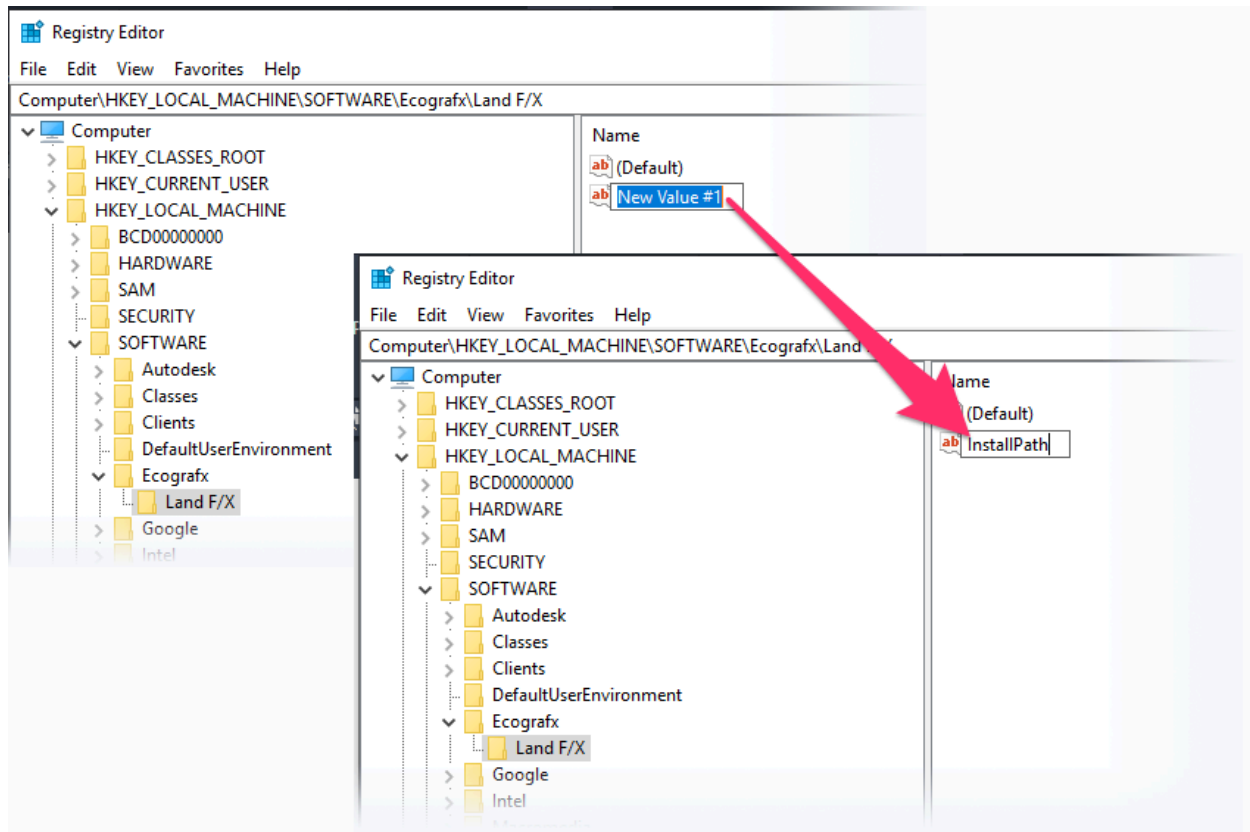
you're following our Manual Installation of the Land F/X Workstation Installer steps, you'd click on **HKEY_LOCAL_MACHINE\SOFTWARE\Ecografx\Land F/X** and then create the string value **InstallPath** key as shown below.

Create a String Value

Creating a string value is similar to creating a key – *right-click* on the key where you want to create the string value, hover on **New** in the menu, and then select **String Value** from the submenu. In this example, we'll create the string value **InstallPath** within the key **Land F/X**.



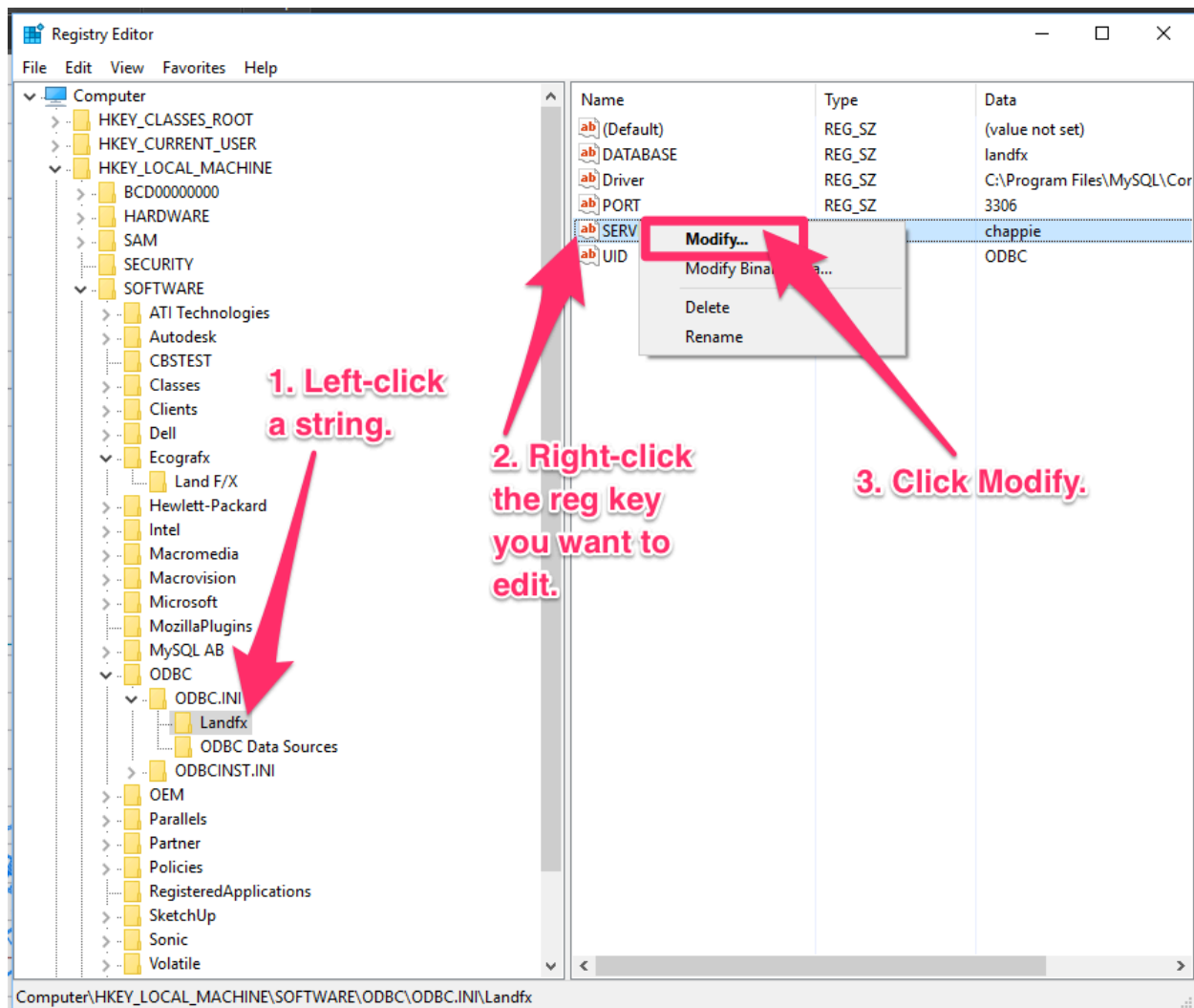
The new string value will appear to the right of the key you right-clicked. Type a name for the new string value (example: **InstallPath**)



Edit a Reg Key or String Value

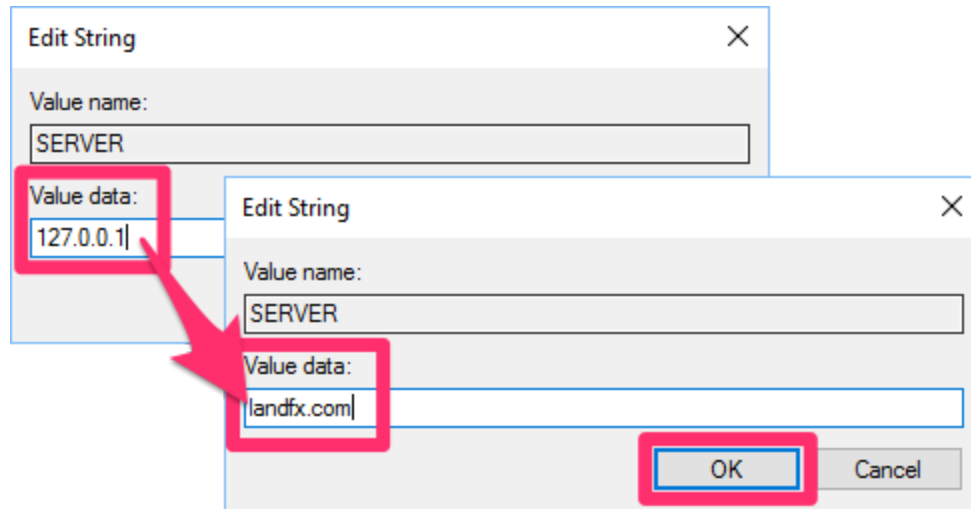
To edit a Reg Key or string value, click the Reg Key folder to open it. In our example, we'll edit the Reg Key

HKEY_LOCAL_MACHINE\SOFTWARE\ODBC\ODBC.INI\LANDFX by clicking the folder **LANDFX** in the path of the same name.



In the right pane, you'll see a list of files, or "strings," contained within the Reg Key you selected. *Right-click* the string you want to edit. Select **Modify** from the menu that opens. In our example, we'll edit the string **SERVER**.

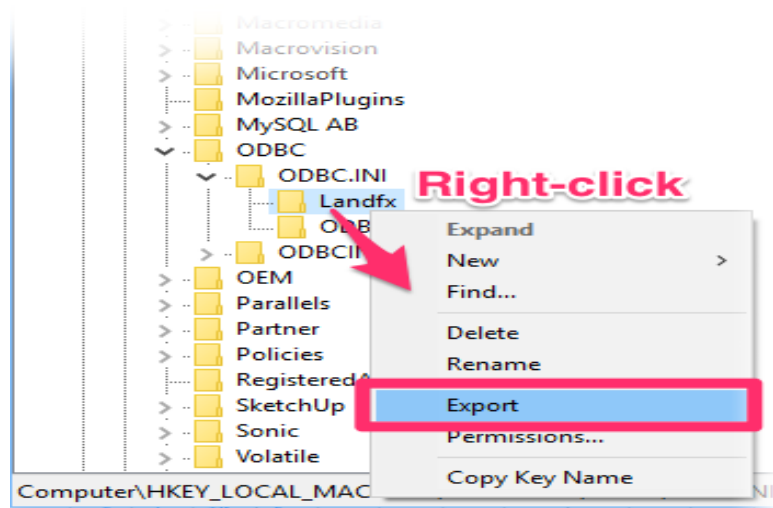
In the **Edit String** dialog box, make your desired change in the **Value data** field. Click **OK** to save your change.



Delete a Reg Key or String Value

Before deleting a Reg Key or string value, it's a good idea to back it up by exporting it. In this example, we'll show you how to export and delete the Reg Key named

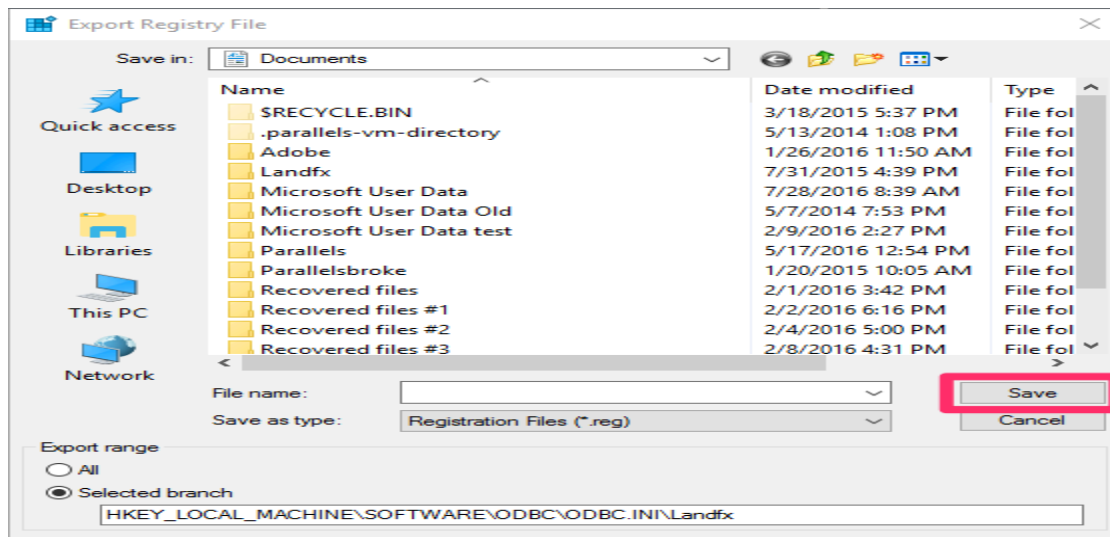
HKEY_LOCAL_MACHINE\SOFTWARE\Classes\Installer\Products\AutoCAD 2015.



Right-click the Reg Key folder, and select **Export** from the menu that opens.

Navigate to the location where you want to back up the Reg Key, and click **Save**.

You'll now have a backed-up copy of the Reg Key, should you need it.



You can now delete the Reg Key safely.

To delete the Reg Key, *right-click* it and select **Delete** from the menu that opens.

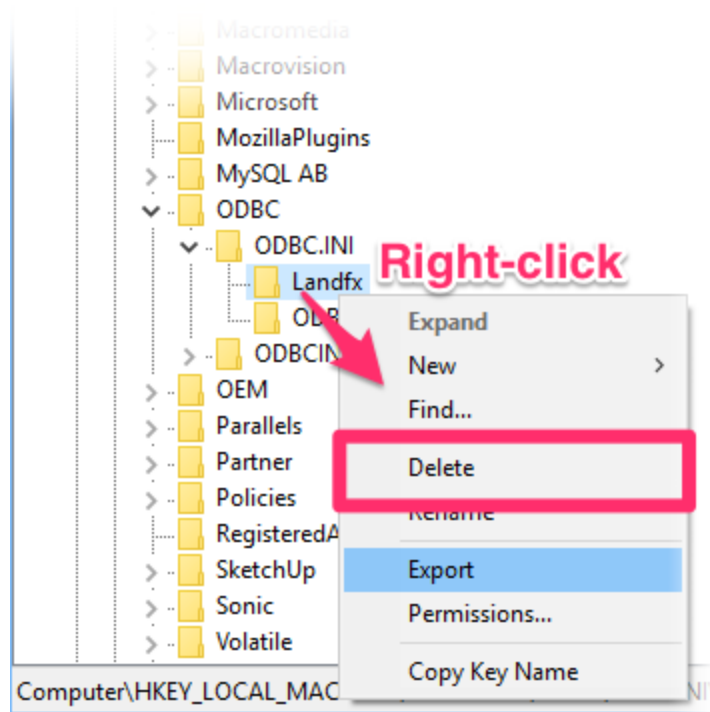
Do not delete this particular Reg Key – or any Reg Key, for that matter – unless specifically instructed to do so. The Reg Key(s) you need to delete will depend on the issue you're experiencing with AutoCAD.

Change Permissions for Reg Keys

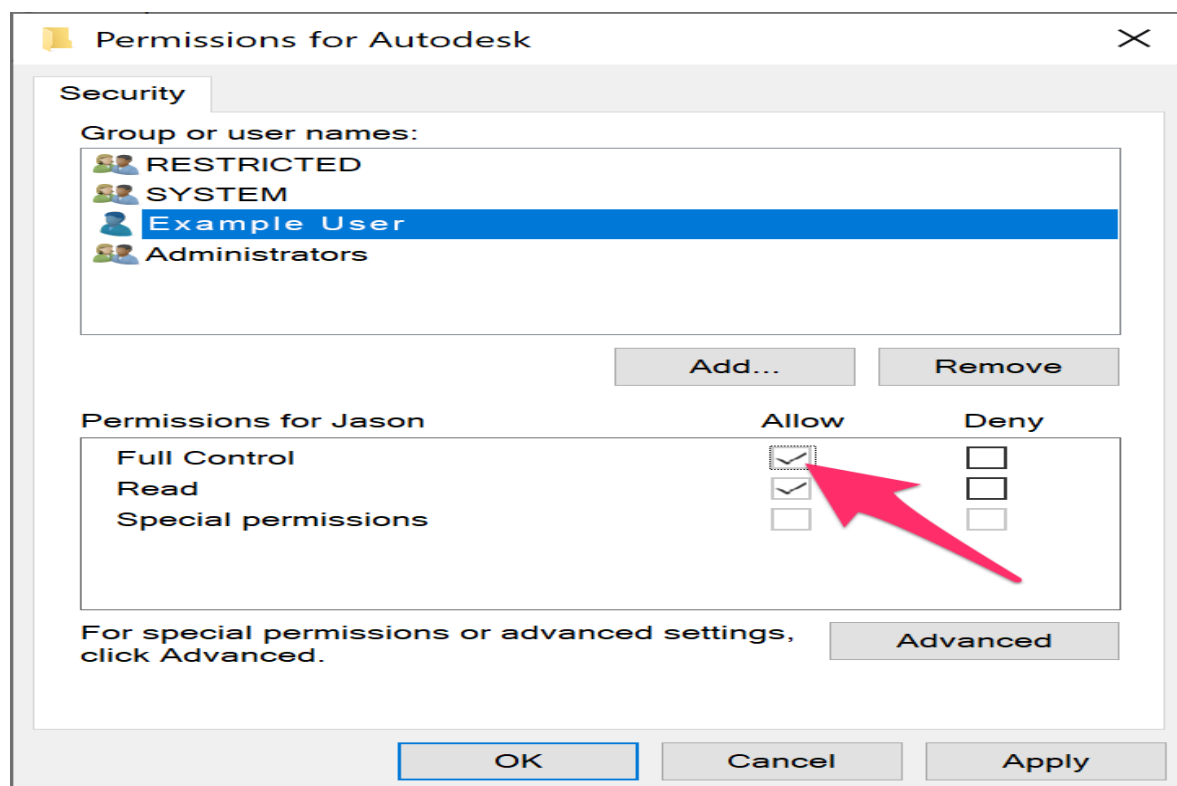
You may also need to change the permissions settings for reg keys. Here's how to do that

Right-click the key whose permissions you need to change.

Select the **Permissions** option from the menu that opens.



Select the user whose permissions you need to change
(example: **HKEY_CURRENT_USER\SOFTWARE\Autodesk.**



Check the box for the permission type you need to grant or deny to the selected user. For example, if you need to grant the user full control (read/write access) for the key, select the **Full control** option in the **Allow** column

Click **Apply** to apply the change, or **OK** to apply the change and close the permissions dialog box.

You'll likely need to grant permission to the **System** and **Administrator** roles as well.

You may need to restart your computer for the permission changes to take effect.

PowerShell for beginners:

PowerShell is a windows command-line shell designed especially for system administrators. It was built on top of the .NET Framework common language runtime (CLR) and the .NET Framework. So, it accepts and returns .NET Framework objects.

PowerShell works in both INTERACTIVE mode and SCRIPT mode. It possesses cmdlet (pronounced "command-let") for doing pre-defined tasks.

Getting Started: Basic PowerShell Commands:

1. Get Help

Retrieves help information for cmdlets, scripts, or functions. It provides details on syntax, parameters, and examples. This cmdlet is essential for beginners and experts to understand and use PowerShell commands effectively.

```
1 Get-Help Get-Process
```

This cmdlet displays detailed information about the `Get-Process` command, including how to use it.

```

PowerShell 7 (x64)
PS C:\> Get-Help Get-Process

NAME
    Get-Process

SYNTAX
    Get-Process [[-Name] <string[]>] [-Module] [-FileVersionInfo]
    [<CommonParameters>]

    Get-Process [[-Name] <string[]>] -IncludeUserName [<CommonParameters>]

    Get-Process -Id <int[]> [-Module] [-FileVersionInfo] [<CommonParameters>]

    Get-Process -Id <int[]> -IncludeUserName [<CommonParameters>]

    Get-Process -InputObject <Process[]> [-Module] [-FileVersionInfo]
    [<CommonParameters>]

    Get-Process -InputObject <Process[]> -IncludeUserName [<CommonParameters>]

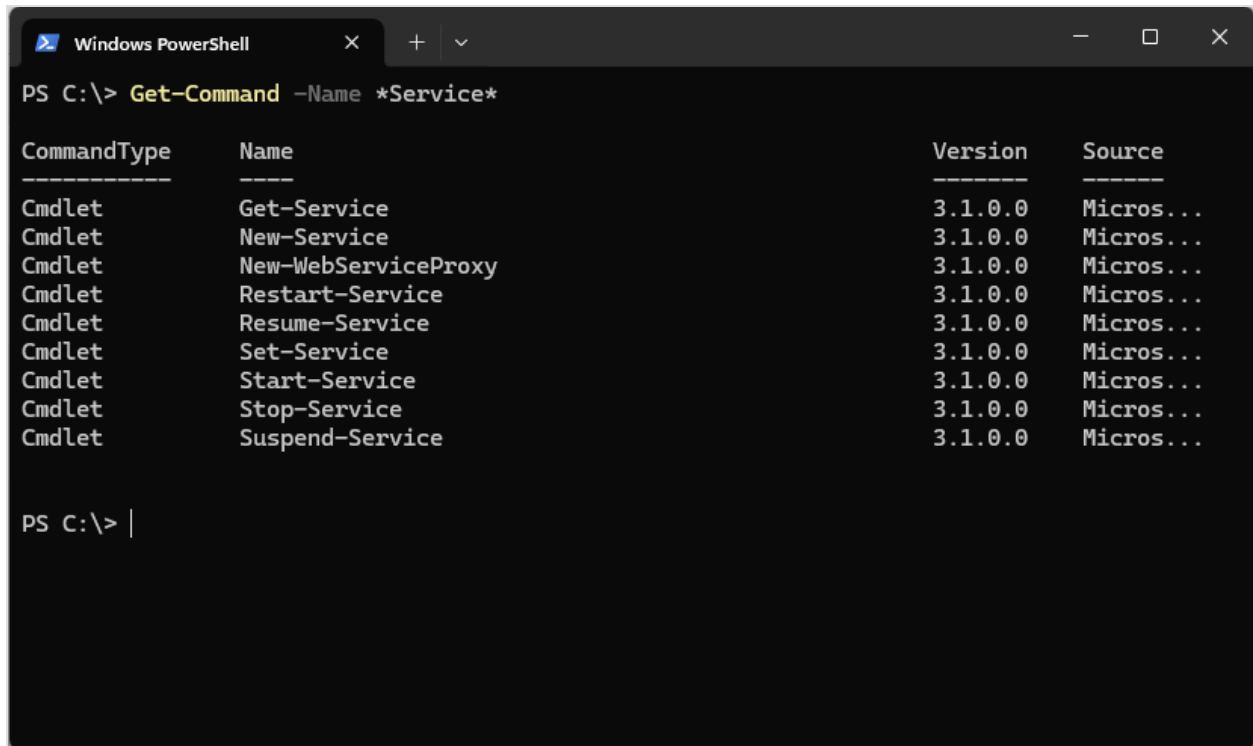
ALIASES
    gps
    ps
  
```

2. Get commands

Lists all available cmdlets, functions, aliases, and applications. It helps users discover available commands and their capabilities. You can filter commands by their verb, noun, or module for better specificity.

```
1 Get-Command -Name *Service*
```

This searches for all commands containing “Service” in their name.



```
Windows PowerShell
PS C:\> Get-Command -Name *Service*

CommandType      Name                               Version      Source
-----
Cmdlet            Get-Service                       3.1.0.0      Micros...
Cmdlet            New-Service                       3.1.0.0      Micros...
Cmdlet            New-WebServiceProxy               3.1.0.0      Micros...
Cmdlet            Restart-Service                   3.1.0.0      Micros...
Cmdlet            Resume-Service                    3.1.0.0      Micros...
Cmdlet            Set-Service                       3.1.0.0      Micros...
Cmdlet            Start-Service                     3.1.0.0      Micros...
Cmdlet            Stop-Service                      3.1.0.0      Micros...
Cmdlet            Suspend-Service                   3.1.0.0      Micros...

PS C:\> |
```

3.write host / write output

Writes output directly to the console. Unlike `Write-Output`, it bypasses the pipeline and is primarily used for display purposes. You can customize the text color and style for better readability.

```
1 Write-Host "Backup Completed!" -ForegroundColor Green
```

Displays a green success message in the console.

The write-output cmdlet is used to send output to the console/file. It's the default behavior of PowerShell to send the specified object to the pipeline, so you often don't need to explicitly use it.

```
1 Write-Output "Task Complete"
```

Passes "Task Complete" to the pipeline for further processing or display.

4.Select Object

The Select-Object cmdlet in PowerShell is used to select specific properties of objects, create new objects with a subset of properties, or limit the number of objects returned from a pipeline. For example:

```
1 Get-ChildItem | Select-Object Name,  
  @{Name='SizeKB';Expression={$_.Length / 1KB}}
```

Creates a new property SizeKB that calculates file size in kilobytes.

5.Clear host

Clears the console screen. It doesn't delete any session data, just removes visual clutter. This is useful for maintaining a clean workspace during complex operations.

```
1 Clear-Host
```

This clears your terminal, giving you a fresh view.

6. Get-Alias / Set-Alias / Remove-Alias

Manages aliases for cmdlets. Aliases provide shortcuts for longer cmdlet names, improving efficiency. You can create, list, and delete aliases as needed.

```
1  
  
2 # Get an existing alias  
  
3 Get-Alias -Name ls  
  
4 # Create new alias  
  
5 Set-Alias ll Get-ChildItem  
  
6 # Remove the alias  
  
7 Remove-Alias ll  
  
8
```

This shows the full command name for the alias `ls`.

7. Get-History / Add-History / Clear-History

Retrieves, appends, or clears command history. This helps track and reuse commands from the current session. Managing history is particularly useful for debugging and session tracking.

Displays a list of all commands you've run in the current session.

```
1 Get-History
```

This lists your recently executed commands, allowing you to easily revisit them.

To delete your command history, ensure privacy or declutter your session.

```
1 Clear-History
```

This removes all previously executed commands from the session history. Here is my article on managing PowerShell history: [Powershell commands history](#)

Advanced Commands for Scripting

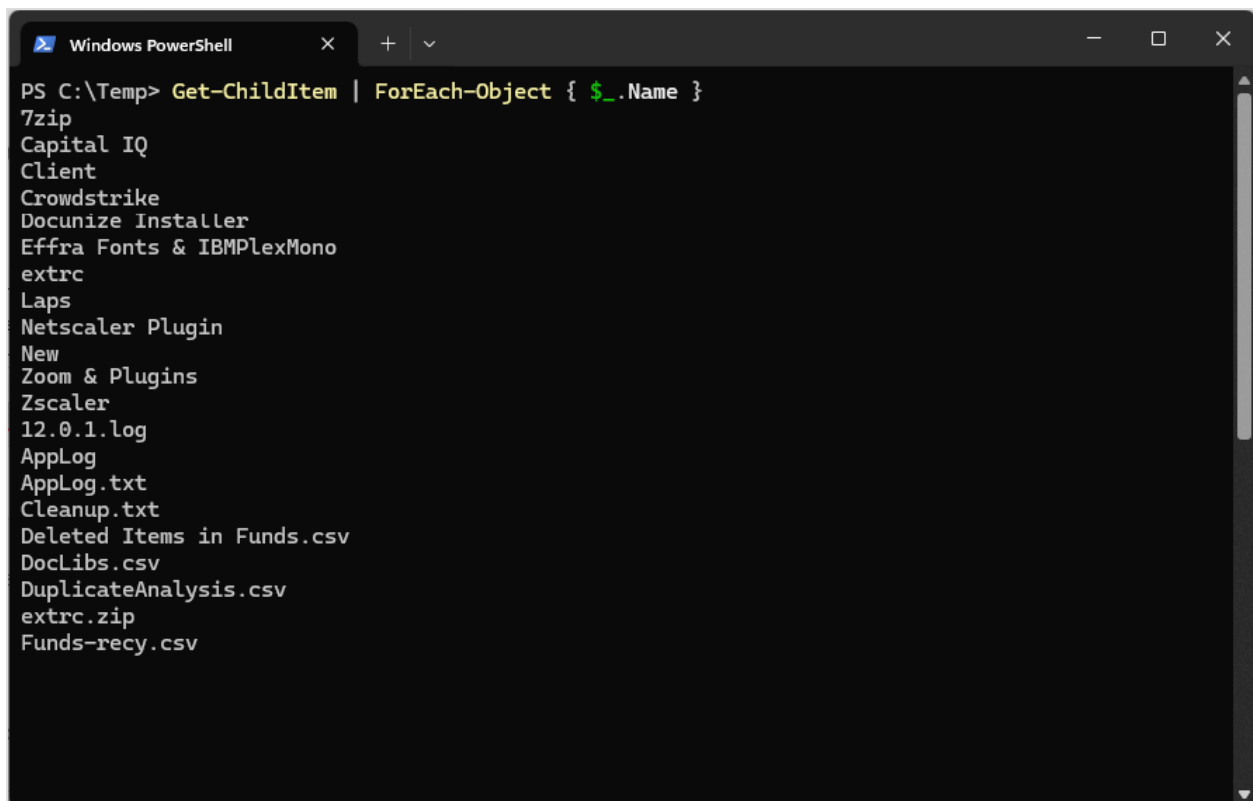
These commands are essential for building robust and efficient PowerShell scripts.

8.For Each OBJECT

Performs operations on each item in a pipeline. It's a versatile cmdlet for iterating over collections and applying transformations or actions to each element.

```
1 Get-ChildItem | ForEach-Object { $_.Name }
```

This lists the names of all files and folders in the current directory.



```
Windows PowerShell
PS C:\Temp> Get-ChildItem | ForEach-Object { $_.Name }
7zip
Capital IQ
Client
Crowdstrike
Docunize Installer
Effra Fonts & IBMPLexMono
extrc
Laps
Netscaler Plugin
New
Zoom & Plugins
Zscaler
12.0.1.log
AppLog
AppLog.txt
Cleanup.txt
Deleted Items in Funds.csv
DocLibs.csv
DuplicateAnalysis.csv
extrc.zip
Funds-recy.csv
```

9. Where-Object

Filters objects based on specified conditions.

```
1 Get-ChildItem | Where-Object { $_.Extension -eq ".txt" }
```

Lists all `.txt` files in the current directory.

10. Start-Transcript / Stop-Transcript

Starts and stops recording a PowerShell session. Transcripts are useful for auditing and documentation. The generated file includes all commands and outputs for later review.

```
1 Start-Transcript -Path "C:\Temp\session.log"
```

This starts recording all commands and outputs to `session.log`.

To stop recording the session and close the transcript file, use:

```
1 Stop-Transcript
```

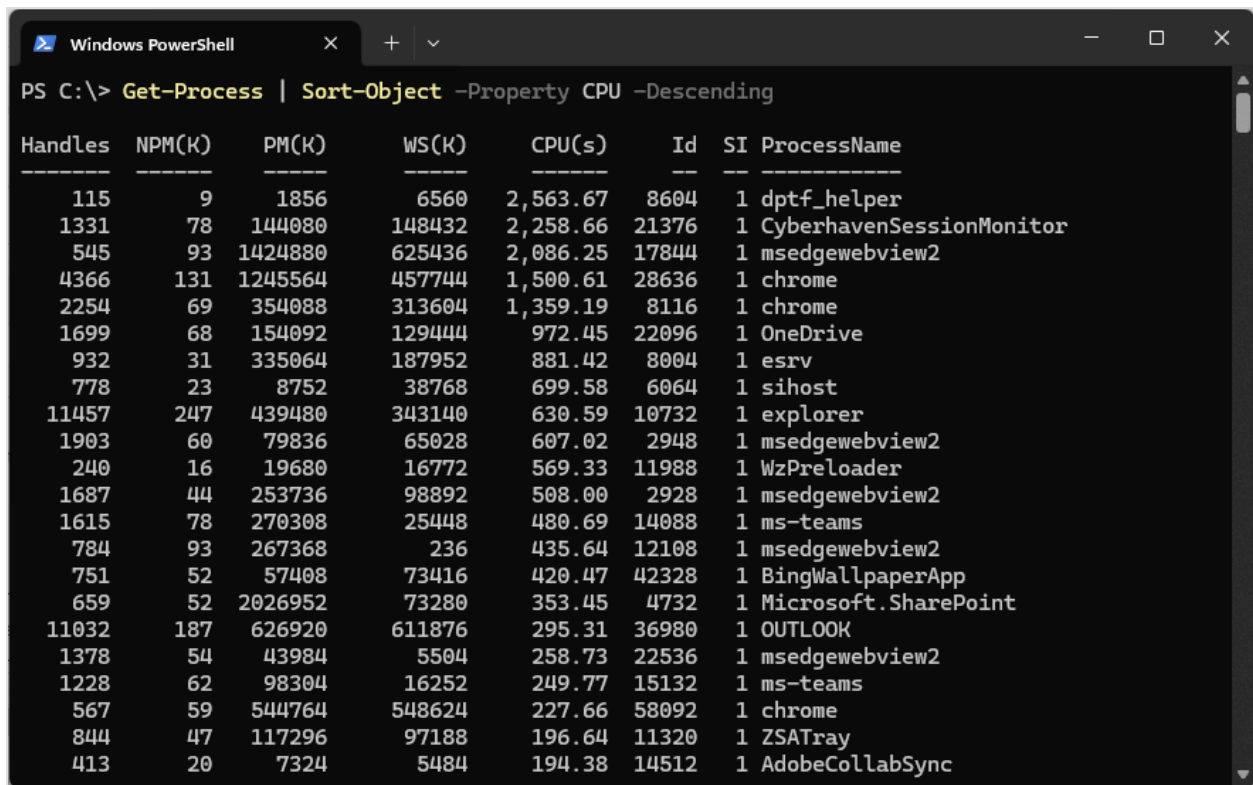
This ends the recording that started with `Start-Transcript`.

11.Sort object

Sorts objects by property values. You can sort in ascending or descending order, and it works with numeric, string, or date values.

```
1 Get-Process | Sort-Object -Property CPU -Descending
```

This sorts processes by CPU usage in descending order.



```
PS C:\> Get-Process | Sort-Object -Property CPU -Descending
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
115	9	1856	6560	2,563.67	8604	1	dptf_helper
1331	78	144080	148432	2,258.66	21376	1	CyberhavenSessionMonitor
545	93	1424880	625436	2,086.25	17844	1	msedgewebview2
4366	131	1245564	457744	1,500.61	28636	1	chrome
2254	69	354088	313604	1,359.19	8116	1	chrome
1699	68	154092	129444	972.45	22096	1	OneDrive
932	31	335064	187952	881.42	8004	1	esrv
778	23	8752	38768	699.58	6064	1	sihost
11457	247	439480	343140	630.59	10732	1	explorer
1903	60	79836	65028	607.02	2948	1	msedgewebview2
240	16	19680	16772	569.33	11988	1	WzPreloader
1687	44	253736	98892	508.00	2928	1	msedgewebview2
1615	78	270308	25448	480.69	14088	1	ms-teams
784	93	267368	236	435.64	12108	1	msedgewebview2
751	52	57408	73416	420.47	42328	1	BingWallpaperApp
659	52	2026952	73280	353.45	4732	1	Microsoft.SharePoint
11032	187	626920	611876	295.31	36980	1	OUTLOOK
1378	54	43984	5504	258.73	22536	1	msedgewebview2
1228	62	98304	16252	249.77	15132	1	ms-teams
567	59	544764	548624	227.66	58092	1	chrome
844	47	117296	97188	196.64	11320	1	ZSATray
413	20	7324	5484	194.38	14512	1	AdobeCollabSync

12.Invoke expression

Executes a string as a command. This cmdlet enables the dynamic execution of commands stored as strings, but it should be used cautiously due to potential security risks.

```
1 Invoke-Expression -Command "Get-Date"
```

This runs the `Get-Date` command.

13.Measured command

Measures the time a command takes to execute. It's particularly useful for performance tuning and benchmarking.

```
1 Measure-Command { Start-Sleep -Seconds 5 }
```

Measures the time taken to pause execution for 5 seconds.

14.Tree object

Sends output to the pipeline and a file. It allows you to save data to a file while continuing to work with it in the pipeline.

```
1 Get-Process | Tee-Object -FilePath "output.txt"
```

This saves process details to `output.txt` and displays them in the console.

File and Folder Management Commands

Efficiently manage your system's files and directories using these cmdlets.

15. Get child item

Retrieves items in a directory. It can list files, directories, or both. It supports recursive search for navigating complex folder structures.

```
1 Get-ChildItem -Path C:\ -Recurse
```

This lists all files and subfolders within the `C:\` directory and its subdirectories. The PowerShell equivalent of the `DIR` command in Windows Command Prompt.

16. Get location

Shows the current directory. This cmdlet is essential for keeping track of your working context in PowerShell.

```
1 Get-Location
```

This prints the directory you're currently in.

17.Set Location

Changes the current directory. It's a quick way to navigate to different folders without using File Explorer.

```
1 Set-Location -Path C:\Scripts
```

Navigates to the `C:\Scripts` folder.

18.Out File

Sends output to a file. It's commonly used for saving logs or reports generated by scripts.

```
1 Get-Process | Out-File -FilePath C:\Processes.txt
```

Saves the list of running processes to `Processes.txt`.

19.New Item

Creates a new file, folder, or registry key. It's a versatile cmdlet for initializing directories or placeholders for data.

```
1 New-Item -Path C:\Temp -ItemType Directory
```

This creates a new folder named `Temp` in the `C:\` directory.

20.Remove Item

Deletes files, folders, or registry keys. It can remove individual files or entire directories recursively.

```
1 Remove-Item -Path C:\Temp -Recurse
```

This deletes the `Temp` folder and all its contents. Please note, everything on given path will be deleted permanently.

21.Rename Item

Renames files or folders. It's useful for organizing or re-labeling data systematically.

```
1 Rename-Item -Path C:\Report.doc -NewName C:\Summary.doc
```

This renames `Report.doc` to `Summary.doc`.

22.Copy Item

Copy files or folders. You can use it for backups or duplicating data structures.

```
1 Copy-Item -Path C:\Data -Destination D:\Backup -Recurse
```

Copies the C:\Data folder to D:\Backup recursively.

23.Move Item

Moves files or folders. It's used for relocating data within or across directories.

```
1 Move-Item -Path C:\Temp\log.txt -Destination C:\backup
2 Move-Item -Path C:\Temp\*.log -Destination D:\Logs
```

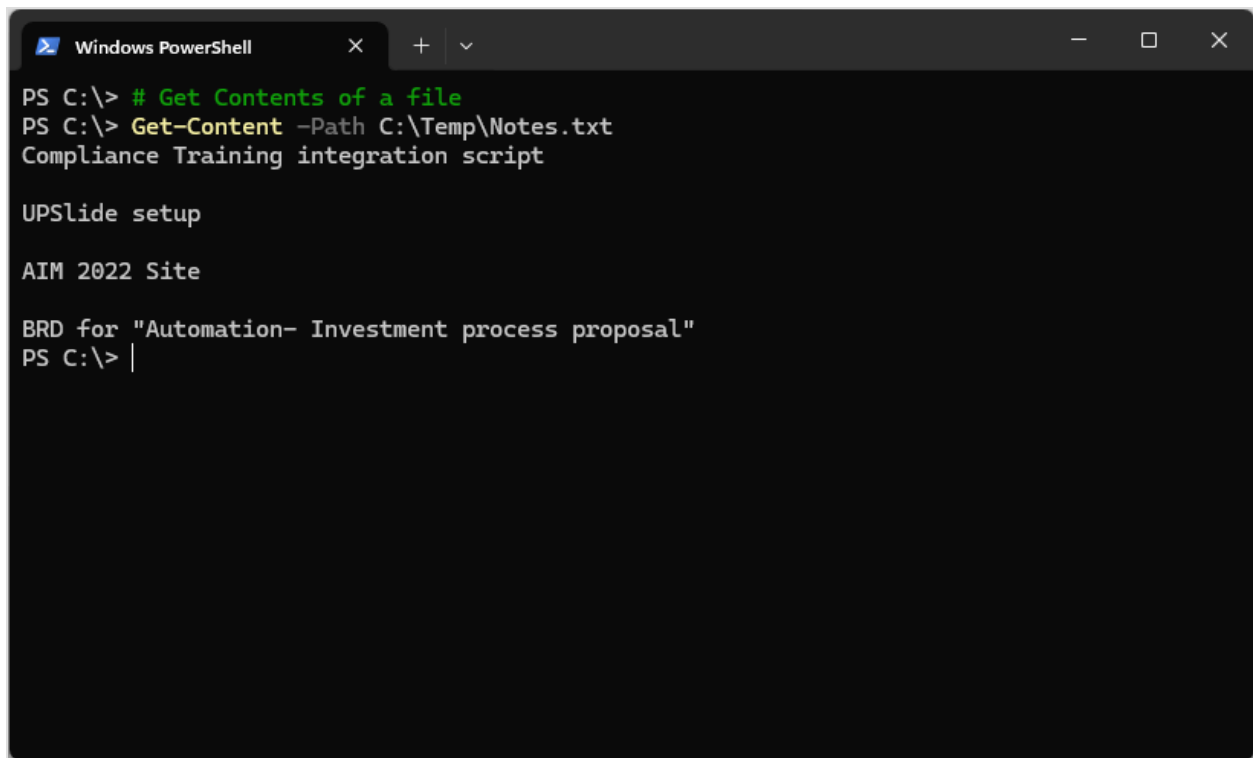
The first command moves Log.txt from Temp to Backup.

24.Get content

Retrieves the content of a file. This cmdlet supports reading large files and can display data line by line.

```
1 Get-Content -Path C:\Temp\Notes.txt
```

This reads the content of `Notes.txt` and displays it on the screen.



```
Windows PowerShell
PS C:\> # Get Contents of a file
PS C:\> Get-Content -Path C:\Temp\Notes.txt
Compliance Training integration script

UPSlide setup

AIM 2022 Site

BRD for "Automation- Investment process proposal"
PS C:\> |
```

25. Set content

Writes content to a file, replacing existing data. It's a straightforward way to initialize or overwrite files.

```
1 Set-Content -Path C:\Notes.txt -Value "Meeting at 3 PM"
```

Replaces the content of `Notes.txt` with "Meeting at 3 PM."

26.Add content

Appends data to a file. It's ideal for adding logs or incremental data to existing files.

```
1 Add-Content -Path C:\Notes.txt -Value "Don't forget the report!"
```

Adds a line to `Notes.txt` without overwriting existing content.

27. Clear-Content

Clears the content of a file. The file remains intact, but all its data is erased.

```
1 Clear-Content -Path C:\OldLog.txt
```

Deletes all text inside `OldLog.txt` without deleting the file itself.

28. Test-Path

Checks whether a path exists. It's commonly used in scripts to ensure prerequisites are met.

```
1 Test-Path -Path C:\Config.xml
```

This checks whether the `Config.xml` file exists in `C:\`. Returns `True` if `Config.xml` exists, otherwise `False`.

29.Join PATH

Combines two or more path components. This cmdlet simplifies working with file paths, especially when handling user input.

```
1 Join-Path -Path C:\Scripts -ChildPath Backup
```

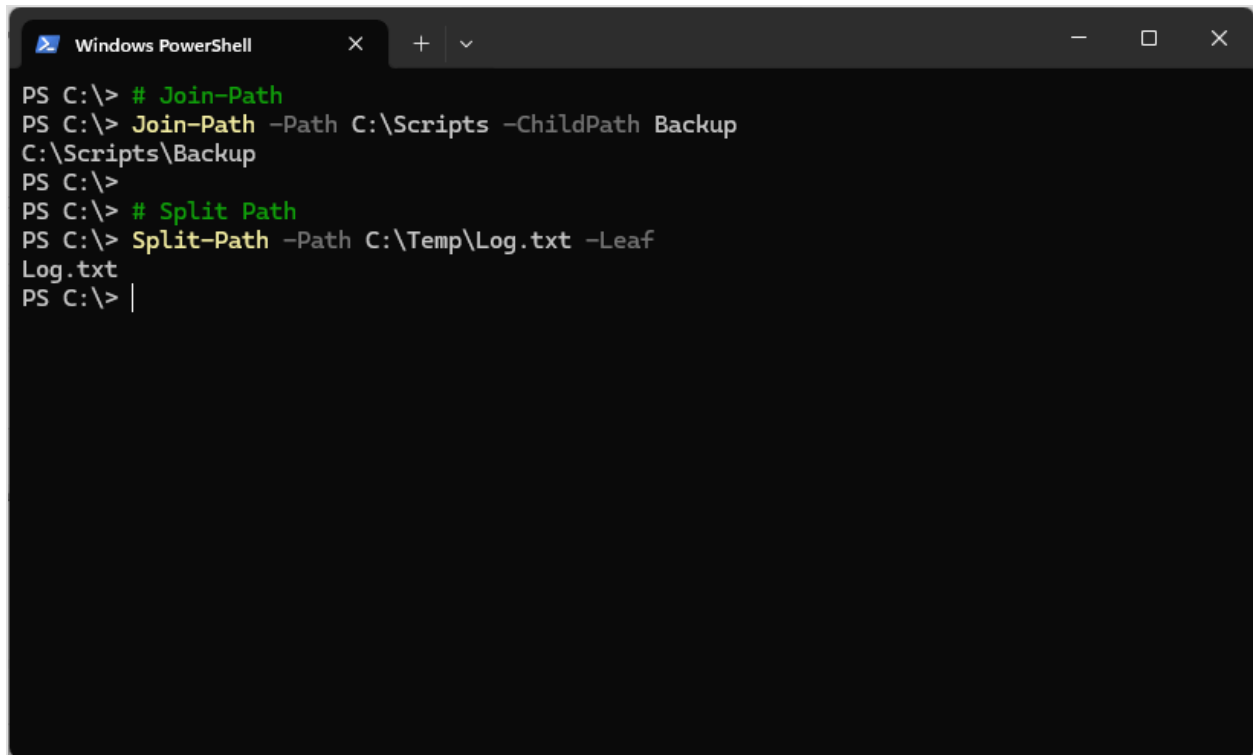
This joins `C:\Scripts` with `Backup` to create a complete path. Outputs `C:\Scripts\Backup`.

30.Split PATH

Splits a path into components. You can extract specific parts like the file name, directory, or parent folder.

```
1 Split-Path -Path C:\Temp\Log.txt -Leaf
```

This extracts the file name (`Log.txt`) from the specified path.



```
Windows PowerShell
PS C:\> # Join-Path
PS C:\> Join-Path -Path C:\Scripts -ChildPath Backup
C:\Scripts\Backup
PS C:\>
PS C:\> # Split Path
PS C:\> Split-Path -Path C:\Temp\Log.txt -Leaf
Log.txt
PS C:\> |
```

Process and Service Management

31.GET PROCESS

Retrieves information about running processes on a local computer. It's a handy tool for monitoring system performance and diagnosing issues. You can filter results by process name or ID.

```
1 Get-Process | Sort-Object -Property CPU -Descending
```

```

PS C:\> Get-Process | Format-Table -Property Name, CPU, ID

Name                                CPU      Id
----                                -
AdobeCollabSync                    229.671875 14512
AdobeCollabSync                    10.609375 22660
AggregatorHost                     3508
ai                                  1.515625 37448
ai                                  0.890625 50396
AppHelperCap                       4616
ApplicationFrameHost               4.203125 11476
armsvc                             5912
audiodg                             0.4375 63092
backgroundTaskHost                  0.0625 5640
backgroundTaskHost                  0.078125 6052
backgroundTaskHost                  0.125 46636
backgroundTaskHost                  0.078125 60584
BingSvc                            76.328125 9504
BingWallpaperApp                   519.3125 42328
CcmExec                            11560
chrome                             1602.53125 8116
chrome                             8.921875 14796
chrome                             12.140625 18668

```

32.Start and Stop process

Starts or stops processes on the system. These cmdlets are useful for automating application launches or terminating unresponsive tasks.

```
1 Start-Process -FilePath "notepad.exe"
```

This launches Notepad. Similarly, the Stop-Process cmdlet terminates a running process by its name or ID.

```
1 Stop-Process -Name notepad
```

This forcefully stops the `notepad` process.

33. Wait Process

Pauses execution until the specified processes have stopped. It's often used in scripts where subsequent actions depend on the completion of specific processes.

```
1 Wait-Process -Name notepad
```

Halts script execution until all Notepad instances are closed.

34. Get-Service

Retrieves and manages Windows services. These cmdlets allow starting, stopping, and restarting services, as well as checking their status.

```
1 Get-Service | Where-Object { $_.Status -eq "Stopped" }
```

Lists all services that are currently stopped.

Start-Service:

```
1 Start-Service -Name "eventlog"
```

Stop-Service

```
1 Stop-Service -Name "iisadmin" -Force -Confirm
```

Restart-Service

```
1 Restart-Service -Name "winmgmt"
```

Data Handling and Reporting Commands

These cmdlets let you process, analyze, and generate reports with your data.

35.Convert To Html

Converts objects into an HTML representation. It's useful for generating simple reports or dashboards.

```
1 Get-Process | ConvertTo-HTML -Title "Process Report" >  
  report.html
```

This creates an HTML report of all running processes and saves it to a file.

36.Export CSV

Exports data to a CSV file. It's commonly used for creating structured, easily shareable data exports.

```
1 Get-Process | Export-Csv -Path "processes.csv"
  -NoTypeInfo
```

This saves process details as a CSV file.

37.Import CSV

Imports data from a CSV file. It transforms CSV rows into PowerShell objects for further processing.

```
1 Import-Csv -Path C:\FileList.csv
```

Reads the content of `FileList.csv` into PowerShell as objects.

38. Select string

Searches for text patterns in files or input strings. It's a powerful tool for parsing and analyzing text.

```
1 Select-String -Path "*.log" -Pattern "Error"
```

This scans all `.log` files in the directory for occurrences of the word "Error."

39.Format Tables

Formats objects as a table for display. It's ideal for presenting data in a structured, readable format.

```
1 Get-Process | Format-Table -Property Name, CPU, ID
```

This presents the name, CPU usage, and ID of all processes in a table format.

40.Format LIST

Formats objects as a list. It's better suited for viewing detailed property values of fewer objects.

```
1 Get-Service | Format-List
```

This shows a detailed list of all services and their properties.

41.Measure Objects

Calculates properties like count, sum, or average from input objects. It's often used for statistical analysis or summaries.

```
1 Get-ChildItem | Measure-Object -Property Length -Sum
```

Calculates the total size of all files in the current directory.

42.Out gridView command

Sends output to an interactive grid window. It's useful for manually filtering and exploring data.

```
1 Get-Process | Out-GridView
```

This opens a grid view in a graphical command window, showing details of all running processes.

43.Group Pojects

Groups objects that contain the same value for a specific property for analysis.

```
1 Get-ChildItem | Group-Object -Property Extension
```

Groups files in the current directory by extension.

44.Write progress

Displays a progress bar in the console. It's commonly used in long-running scripts to provide user feedback.

```
1 For ($i=1; $i -le 10; $i++) {  
2     Write-Progress -Activity "Processing" -Status "Step $i of 10"  
   -PercentComplete ($i * 10)  
3     Start-Sleep -Seconds 1  
4 }
```

This shows a progress bar as the loop executes.

Module Management Cmdlets in PowerShell

Modules in PowerShell are collections of cmdlets, functions, variables, and resources that help extend PowerShell's functionality. Managing modules effectively allows you to add, update, and organize tools for specific tasks.

45. Get Module

This lists all available PowerShell modules. It helps users discover installed modules and their details.

```
1 Get-Module -ListAvailable
```

Displays all modules installed on the system, both loaded and unloaded.

46.Import Modules

Loads a PowerShell module into the current session. It's necessary to use cmdlets from specific modules.

```
1 Import-Module ActiveDirectory
```

This imports the Active Directory module, enabling you to manage AD objects.

47.INSTALL MODULES

Installs PowerShell modules from the PowerShell Gallery.

```
1 Install-Module -Name Sqlserver
```

This installs the SQL Server module for testing scripts. To install a module for the current user:

```
1 Install-Module -Name Sqlserver -Scope CurrentUser
```

48.Update Modules

Updates installed modules to their latest versions. It ensures that you're using the most recent features and fixes.

```
1 Update-Module -Name Pester
```

This updates the `Pester` module if a new version is available.

49. Uninstall Modules

The `Uninstall-Module` cmdlet in PowerShell is used to remove installed modules from your system. Here's how you can use it:

```
1 Uninstall-Module -Name <ModuleName>
```

Use the `Uninstall-Module` cmdlet, specifying the name of the module you want to remove.

50. Add PSSnapin

Adds a snap-in to the current session. Snap-ins extend PowerShell with additional cmdlets and providers. This cmdlet is used primarily in older PowerShell versions.

```
1 Add-PSSnapin Microsoft.SharePoint.PowerShell
```

As you become more comfortable with these basic commands, you'll naturally progress to more advanced operations.

PowerShell Cmdlets for Registry Management

51. Get-Item

Retrieves a specified item, such as a file, folder, or registry key. This cmdlet is a general-purpose tool for accessing various resources in PowerShell.

```
1 Get-Item -Path C:\Windows\System32\notepad.exe
```

Retrieves information about the Notepad executable file.

52. New-ItemProperty, Set-ItemProperty Remove-ItemProperty Commands

Manages item properties, such as registry keys. These cmdlets are crucial for configuring systems and applications via PowerShell.

```
1 New-ItemProperty -Path "HKLM:\Software\MyApp" -Name "Version"  
-Value "1.0"
```

Creates a new registry property named `Version` with a value of `1.0` under `MyApp`.

Set-ItemProperty

```
1 Set-ItemProperty -Path "HKLM:\Software\Crescent" -Name "Version"  
   -Value "2.0"
```

Remove-ItemProperty

```
1 Remove-ItemProperty -Path "HKLM:\Software\Crescent" -Name  
   "Version"
```

System Information Commands

These cmdlets provide details about the system and user environment.

53.Get Date

Displays the current date and time in various formats.

```
1 Get-Date
```

This prints the current date and time to the console.

54. Get Computer Info

Displays comprehensive system details, including OS, hardware, and more.

```
1 Get-ComputerInfo
```

This retrieves detailed information about the computer

55. Restart and switch Computer

Restarts the local or remote computer. This cmdlet is useful for applying updates or changes requiring a reboot

```
1 Restart-Computer
```

56. Get-Clipboard Or Set-Clipboard

Interacts with the Windows clipboard. These cmdlets allow for programmatic copying and pasting of text or objects.

```
1 Get-Clipboard
```

This displays whatever text or data is copied to the clipboard.

Set-Clipboard

Sets new content for the clipboard

```
1 "Copied text" | Set-Clipboard
```

This copies “Copied text” to the clipboard.

57. Get-Host

Displays information about the current PowerShell host.

```
1 Get-Host
```

This shows details like the Powershell version and UI settings.

58. New Guid

Generates a new globally unique identifier (GUID). This is often used in scripting for creating unique IDs.

```
1 New-Guid
```

Outputs a new GUID, such as 8d4d7607-1e4b-4a1b-84ec-b50c7ff1ea16.

Network and Remote Management Commands

PowerShell makes it easy to manage remote systems and network configurations. These commands are essential for IT pros working across multiple machines or networks.

59. Test-Connection / Test-NetConnection

Tests network connectivity to a host, similar to `ping`.

```
1 Test-Connection -ComputerName google.com -Count 3
```

This checks if the `google.com` server is reachable and sends three requests. The `Test-NetConnection` combines the functionality of several network tools into one versatile command.

```
1 Test-NetConnection -ComputerName example.com -Port 80
```

60. Get IP Address Net

Retrieves IP configuration details of the system's network adapters. It provides information such as IP address, subnet, and gateway.

1 `Get-NetIPAddress`

This shows all IP addresses assigned to your network interfaces.

Here are some key best practices I've learned throughout my career:

1. **Use Clear and Descriptive Names:** Choose meaningful names for your variables, functions, and scripts. This makes your code more readable and self-documenting.
2. **Comment Your Code:** While PowerShell's syntax is relatively readable, adding comments to explain complex logic or the purpose of specific code blocks can be invaluable, especially when revisiting code months later.
3. **Use Splatting for Complex Commands:** When dealing with commands that have many parameters, use splatting to improve readability.
4. **Leverage PowerShell's Built-in Help System:** Before creating a custom function, check if there's already a cmdlet that does what you need. Use `Get-Help` to explore existing cmdlets.
5. **Use Error Handling:** Implement **try/catch blocks** to handle errors gracefully and provide meaningful error messages.
6. **Follow the PowerShell Verb-Noun Naming Convention:** When creating your own functions, stick to the standard verb-noun naming convention (e.g., `Get-UserInfo`, `Set-NetworkConfiguration`).
7. **Use PowerShell Modules:** Organize your functions into modules for better code management and reusability.

-
8. Avoid Using Aliases in Scripts: While aliases are great for interactive use, they are not useful in scripts for better readability and portability.

Conclusion

Windows Internals means understanding what happens inside Windows after you open an application, access a file, connect to a network, or log in.

Windows is built in layers. Applications run in **user mode** with limited permissions, while the Windows kernel, drivers, memory manager, and I/O components run in **kernel mode** with high privileges. This separation helps keep the operating system stable and secure.

For cybersecurity, Windows Internals is important because attacks, malware, detection tools, and forensic investigations all interact with these core components: processes, threads, memory, services, registry, file system, tokens, and event logs. Understanding the normal working of Windows helps you identify abnormal behavior.

In short: do not learn Windows Internals only as theory. Use it to understand how Windows behaves on a real machine, how attackers may misuse legitimate features, and how defenders can detect that misuse. The Windows Internals reference covers core Windows architecture and internal components in detail.